

2026-06-06 14:52:35

You Asked 

 SWE 4603-Week 3 Part 1.pdf

read this. dont say anything

 Gemini 3.5 Flash

Acknowledged. I have thoroughly reviewed the provided lecture slides on **Software Testing and Quality Assurance (SWE 4603)** from the Islamic University of Technology (IUT).

The material covers **Dynamic Black-Box Testing Techniques**, including:

- **Boundary Value Analysis (BVA):** Boundary Value Checking ($4n + 1$), Robustness Testing ($6n + 1$), Worst-Case Testing (5^n), and Robust Worst-Case Testing (7^n).
- **Equivalence Class Testing:** Weak Normal, Strong Normal, Weak Robust, and Strong Robust Partitioning.
- **State Table-Based Testing:** Converting Finite State Machines (FSMs) and State Graphs into State Tables and Test Cases.
- **Decision Table-Based Testing:** Modeling complex combinations of input conditions (Condition Stubs) to determine system actions (Action Stubs).

I will remain silent and await your specific instructions, questions, or tasks regarding this material.

2026-06-06 15:04:58

You Asked 

explain page 5 in details, in easy way and properly.

 Gemini 3.5 Flash

Based on the file "SWE 4603-Week 3 Part 1.pdf", page 5 covers the foundational concept of **Black-Box Testing**. Let's break it down into simple, easy-to-understand terms.

What is Black-Box Testing?

Imagine you buy a brand-new microwave. You don't need to know how the internal circuit board is wired or how the magnetron generates micro-radiation waves to use it. You only care about this: *If I press the "Popcorn" button (Input), does it pop my popcorn perfectly without burning it (Expected Output)?*

That is exactly what **Black-Box Testing** is.

- **The "Box" is the software:** The term "black box" signifies that the inner workings, structure, and code of the system are completely hidden or invisible to the tester.
- **The Focus:** A tester only focuses on what goes **in** (Input test data) and what comes **out** (Output results). They are checking if the system behaves correctly based on its **functional requirements** (what the software is *supposed* to do).

The Core Concept: Input vs. Output

The page illustrates a diagram showing how a black-box test operates:



You don't care *how* the program computes the results or what programming language was used; you just care that the math or logic is correct.

Real-World Example: User Acceptance Testing (UAT)

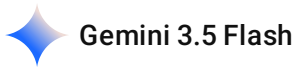
Page 5 gives an excellent real-world application of this technique: **User Acceptance Testing (UAT)**.

- **Who does it?** Usually, the client, end-users, or specialized QA testers.
- **How do they do it?** They log into the software and try to use it like a normal customer would.
- **Why is it Black-Box?** These users have zero access to the source code. They do not know how the system was implemented behind the scenes. Their only goal is to verify that the software meets their business requirements and works properly when they type in typical, real-world data.

Summary Summary

- **You cannot see inside:** The internal code or system design is completely hidden.
- **Requirement-driven:** It checks *what* the software does against the rules, not *how* it does it.
- **Simple check:** Input → System → Does the Output match expectations?

explain page 8 in details, in easy way and properly. and remember to follow the exact page serial of the pdf. like you did with page 5. ignore if there are and page number on the slides.



Based on the slide sequence in the document, **Page 8** outlines the main goals of Black-Box Testing. It breaks down the specific types of errors, bugs, and issues that this testing technique is explicitly designed to uncover.

Let's look at each of these five categories in detail, explained simply with real-world analogies:

1. Incorrect or Missing Functions

The Slide says: *"To test the functional validity of the software so that incorrect or missing functions can be recognized"*

- **What it means:** This is about checking if the software actually does what it promised to do in the requirements manual.
- **Easy Example:** Imagine you buy a smartphone that advertises a "Dark Mode" feature. If you switch it on and the screen stays bright white, that is an **incorrect function**. If you open the settings menu and the "Dark Mode" button isn't even there, that is a **missing function**. Black-box testing uncovers these gaps by trying out the features from an outsider's perspective.

2. Interface Errors

The Slide says: *"To look for interface errors"*

- **What it means:** An interface is how two things communicate. This could be the **User Interface (UI)** (how a human interacts with the screen) or an **Application Programming Interface (API)** (how data moves from one system component to another).
- **Easy Example:** You are on an e-commerce checkout page. You click the "Pay Now" button, but nothing happens because the button isn't properly connected to the payment system. Or perhaps you type your name into a form, but the text box truncates it because it's too small. These are interface errors. Black-box testing catches them by checking how data enters, travels through, and exits the boundaries of the system.

3. System Behavior and Performance Errors

The Slide says: *"To test the system behavior and check its performance"*

- **What it means:** Software needs to be more than just accurate; it needs to be well-behaved, stable, and responsive. Performance testing looks at how the system acts under normal conditions.
- **Easy Example:** Imagine a banking app where clicking "View Statement" works perfectly, but takes 45 seconds to load the page. The function isn't missing or broken, but its *behavior and performance* are unacceptable. Black-box testing evaluates these external quality attributes like response times and system transitions.

4. Maximum Load or Stress Errors

The Slide says: *"To test the maximum load or stress on the system"*

- **What it means:** This checks how the software behaves when pushed to its absolute limits (Load and Stress Testing).
- **Easy Example:** Think of a university result publication website. It works perfectly when 5 students are online. But what happens at 4:00 PM when 50,000 students log in simultaneously to check their grades? Does the website slow down gracefully, or does the entire server crash and throw an error page? Black-box testing simulates these extreme external scenarios to find breaking points.

5. Acceptance Limits (User/Customer Acceptance)

The Slide says: *"To test the software such that the user/customer accepts the system within defined acceptable limits"*

- **What it means:** At the end of the day, the software must satisfy the end user's expectations and legal/business constraints. This ensures the software fits into the client's practical workflow.
- **Easy Example:** A company hires a developer to build an automated billing system that must process invoices in less than 3 seconds to prevent long queues. If the final software takes 10 seconds per invoice, it violates the client's "acceptable limits" and will be rejected. Black-box testing validates that the product fulfills these final contract boundaries so the customer can happily accept it.

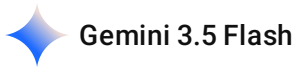
Summary Checklist of Page 8

Black-box testing acts like a final quality control inspector who asks:

1. Does it do everything it's supposed to do? (Functions)
2. Do the buttons, screens, and connections work smoothly? (Interfaces)
3. Is it fast and stable enough during normal use? (Performance)
4. Does it survive a massive rush of users? (Stress/Load)
5. Will the client actually be happy to pay for this? (Acceptance)



explain page 10 in details, in easy way and properly.



Based on the exact page sequence of the file "SWE 4603-Week 3 Part 1.pdf", **Page 10** introduces the core concept of **Boundary Value Analysis (BVA)**.

This is one of the most critical and widely used black-box testing techniques. Let's break down the slide's contents in detail, using simple explanations and visuals.

1. Why Test the Boundaries?

The Slide says: "Test cases designed with boundary input values have a high chance to find errors. It means that most of the failures crop up due to boundary values."

- **The Core Idea:** Programmers are human, and when they write code, they make the most mistakes at the **edges** (boundaries) of input ranges.
- **The "Off-by-One" Mistake:** A developer might accidentally write `if (age > 18)` instead of `if (age >= 18)`. This tiny typo completely breaks the system's logic, but *only* for the exact number 18. Testing numbers right in the middle (like 25 or 40) won't catch this bug. Testing the boundary values will.

2. What is a Boundary?

The Slide says: "Boundary means the maximum or minimum value taken by the input domain."

If a system accepts values within a specific range, the **minimum allowable value** and the **maximum allowable value** are your boundaries.

3. Detailed Example Breakdown

The slide provides a visual graph mapping two integer variables, **A** and **B**, to explain how we pick boundary test points.

Variable A (Range: 10 to 255)

- Variable A must be an integer between 10 and 255.

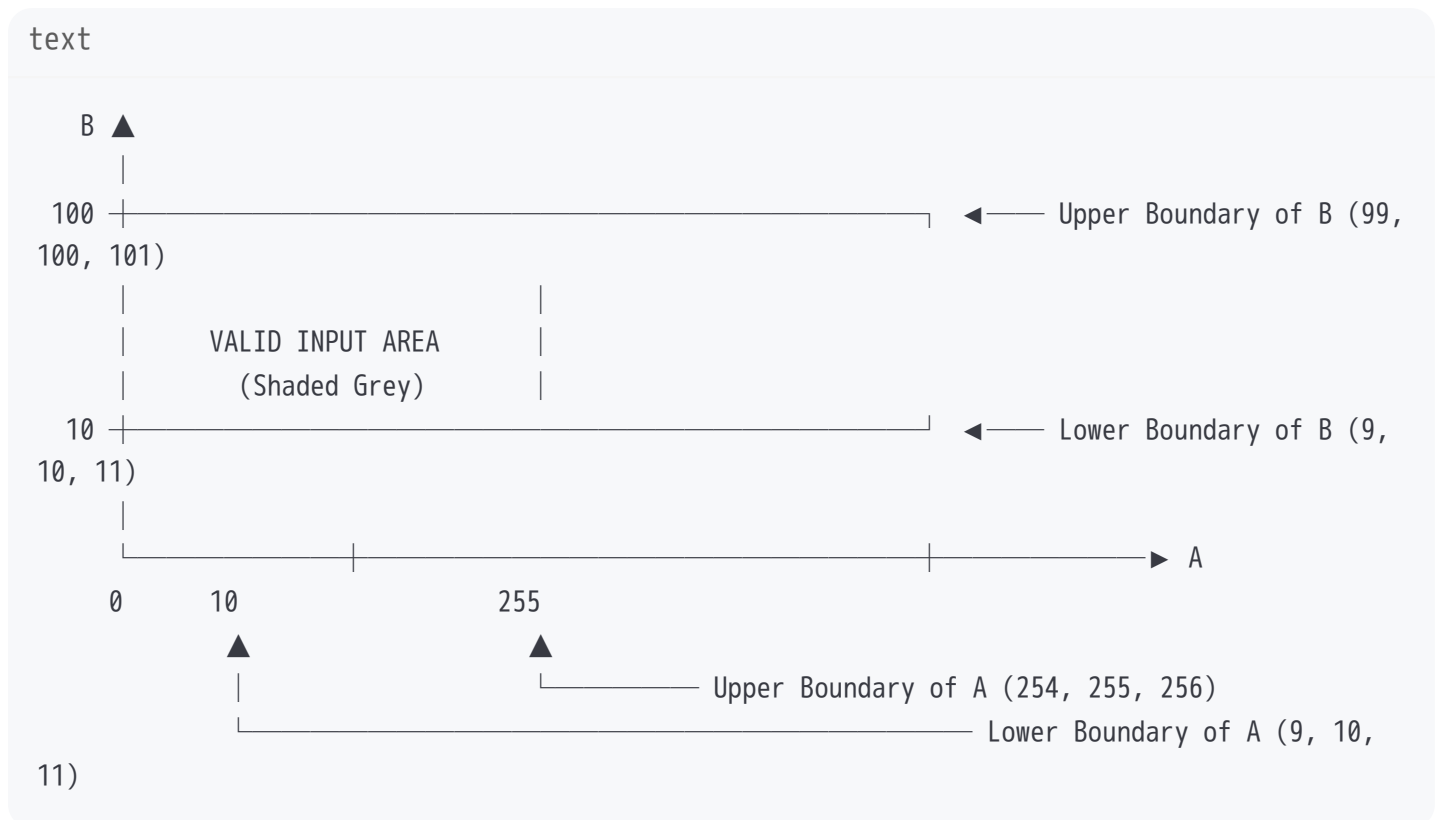
- To check the lower boundary (**10**), BVA tells us to test three specific points around it: **9, 10, and 11**.
- To check the upper boundary (**255**), we test three points around it: **254, 255, and 256**.

Variable B (Range: 10 to 100)

- Variable B must be an integer between 10 and 100.
- Checking the lower boundary (**10**) means testing: **9, 10, and 11**.
- Checking the upper boundary (**100**) means testing: **99, 100, and 101**.

4. Visualizing the Testing Graph on Page 10

The chart on page 10 plots Variable A on the horizontal axis (X -axis) and Variable B on the vertical axis (Y -axis). The shaded grey rectangle represents the **Valid Input Domain** where the software should function normally:



Summary of the BVA Strategy on Page 10

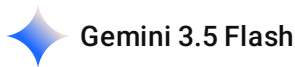
Instead of picking random numbers to test, page 10 teaches you to look at the maximum and minimum limits of your inputs. For any boundary value X , you should always test:

1. **The exact boundary:** X
2. **Just below the boundary:** $X - 1$
3. **Just above the boundary:** $X + 1$

By doing this, you drastically increase your chances of catching sneaky programming bugs right where they love to hide.

2026-06-06 15:18:31 You Asked 

explain page 11 and 12 in details, in easy way and properly.



Here is the detailed, easy-to-understand breakdown of **Page 11** and **Page 12** from your slides, following the exact serial sequence of the PDF.

Page 11: Boundary Value Analysis (BVA) Methods

Page 11 serves as an index or menu page. It introduces the four distinct sub-techniques or frameworks that BVA offers to design test cases.

When you are testing software using boundaries, you don't just guess random edge numbers; you follow one of these four formal methods:

- **1. Boundary Value Checking (BVC):** This is the basic, standard method. It is also widely known as **Normal Boundary Value Testing**. It only focuses on *valid* values within the range.
- **2. Robustness Testing Method:** This is an extension of BVC. The word "robust" means a system's ability to handle mistakes. Therefore, this method tests what happens when you push the system *past* its limits into *invalid* territory (e.g., entering an age of -1 or 101).
- **3. Worst-Case Testing Method:** This method assumes that multiple variables might break or hit boundaries at the exact same time (a multi-fault assumption).
- **4. Robustness Worst-Case Testing Method:** This is the ultimate, most extreme combination of methods 2 and 3. It checks for multiple faults while simultaneously throwing invalid/out-of-range values at the system.

Page 12: BVA: Boundary Value Checking (BVC)

Page 12 zooms in on the very first method: **Boundary Value Checking (BVC)**. Let's break down its core operational rules:

A. Core Concept & The "Single Fault Assumption"

The Slide says: "The test cases are designed by holding one variable at its extreme value and other variables at their nominal values in the input domain. (Single Fault Assumption)"

- **What it means:** BVC operates on the theory that program failures rarely happen because two things broke simultaneously. Instead, they usually happen because of a *single* mistake.
- **How you test:** Because of this "Single Fault" rule, if you are testing a system with two inputs (Variable X and Variable Y), you will test the boundaries of Variable X while keeping Variable Y safely at a normal, average value (**Nominal value**). Then, you flip it: you test the boundaries of Variable Y while keeping Variable X normal. You *never* test boundaries for both variables at the exact same time in this method.

B. Valid Values Only

The Slide says: "Concerned with valid values of the input variables."

- **What it means:** BVC is gentle. It never inputs data that is completely illegal or out of bounds. Every single number you pass to the program will be within the technically accepted range.

C. The 4 Extreme Testing Points

The Slide says: "The variable at its extreme value can be selected at: Min, Min+, Max, Max-"

To test a single variable's boundaries using BVC, you look at its input range and select exactly four extreme test values:

1. **Minimum Value (Min):** The absolute lowest valid boundary number allowed.
2. **Value just above the minimum value (Min+):** The step right inside the lower boundary.
3. **Maximum Value (Max):** The absolute highest valid boundary number allowed.
4. **Value just below the maximum value (Max-):** The step right inside the upper boundary.

Real-World Analogy for Page 12

Imagine an online system that allows you to buy tickets, where you must select a quantity between **1 and 10**.

According to Page 12's BVC rules, your extreme testing points for ticket quantities are:

- Min = 1
- Min+ = 2
- Max = 10
- Max- = 9

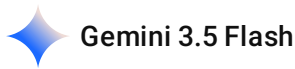
Notice that you do *not* test 0 or 11 here. Why? Because BVC only concerns itself with **valid values** inside the range! Testing 0 and 11 is saved for the *Robustness Testing* method coming up next in

your slides.

2026-06-06 15:28:38

You Asked 

explain page 13 and 14 in details, in easy way and properly.



Here is the detailed, easy-to-understand breakdown of **Page 13** and **Page 14** from your slides, which visualizes how test cases are mathematically calculated and mapped out.

Page 13: BVA: Boundary Value Checking (Mathematical Formulation)

Page 13 takes the standard **Boundary Value Checking (BVC)** concepts from Page 12 and shows you exactly how to apply them to an actual test suite with multiple variables.

A. The 2-Variable Example Mapping

Imagine you are testing a module with two input variables: **A** and **B**.

- Let's say the valid range for **A** is vertical (Y -axis), bounded by values **c** and **d**.
- The valid range for **B** is horizontal (X -axis), bounded by values **a** and **b**.

Remember the **Single Fault Assumption** from Page 12? It says we only test boundaries for *one* variable at a time while holding the other variable at its normal, middle value (**Nominal** or X_{nom}).

Following this rule, the slide lists the **9 exact test cases** you can design:

- **Testing Variable B's Boundaries** (Variable A stays safely at A_{nom}):
 1. A_{nom}, B_{min} (Normal A, Minimum valid B)
 2. A_{nom}, B_{min+} (Normal A, Just above minimum B)
 3. A_{nom}, B_{max} (Normal A, Maximum valid B)
 4. A_{nom}, B_{max-} (Normal A, Just below maximum B)
- **Testing Variable A's Boundaries** (Variable B stays safely at B_{nom}):
 5. A_{min}, B_{nom} (Minimum valid A, Normal B)
 6. A_{min+}, B_{nom} (Just above minimum A, Normal B)
 7. A_{max}, B_{nom} (Maximum valid A, Normal B)
 8. A_{max-}, B_{nom} (Just below maximum A, Normal B)
- **The Baseline Center Point**: 9. A_{nom}, B_{nom} (Everything is perfectly normal and average)

If you look at the dot diagram on Page 13, it forms a perfect **cross shape** (+) because the points only move along the middle axes and never into the corners!

B. The Generalized Formula

The Slide says: "It can be generalized that for n variables in a module, $4n + 1$ test cases can be designed with boundary value checking method."

If a module has n input variables, you don't need to manually guess how many test cases you need. Just use the formula:

$$\text{Total BVC Test Cases} = 4n + 1$$

- **Why?** Each variable requires 4 boundary checks (Min, Min+, Max, Max-). If you have n variables, that's $4 \times n$ tests. Finally, you add $+1$ for the absolute central nominal baseline case (A_{nom}, B_{nom}).
- *Example:* For 2 variables ($n = 2$), $4(2) + 1 = 9$ test cases.

Page 14: BVA: Robustness Testing Method

Page 14 steps up the game by introducing **Robustness Testing**. While BVC only handles *valid* values, Robustness testing forces the system to face **invalid inputs** to see if it handles them gracefully without crashing.

A. Extending the Extreme Points

The Slide says: "It is an extension of BVC such that boundary values are exceeded as... Max+ and Min-"

To make your test robust, you take the 9 existing valid test cases from BVC and add **out-of-bounds** steps:

1. **A value just greater than the Maximum value ($Max+$):** Stepping 1 unit *outside* the upper limit.
2. **A value just less than the Minimum value ($Min-$):** Stepping 1 unit *outside* the lower limit.

Following our 2-variable example (A and B), we add 4 new invalid test cases to our suite:

- **Invalid tests for Variable A** (B stays nominal):
 - 10. A_{max+}, B_{nom} (Too high A, Normal B)
 - 11. A_{min-}, B_{nom} (Too low A, Normal B)
- **Invalid tests for Variable B** (A stays nominal):
 - 12. A_{nom}, B_{max+} (Normal A, Too high B)
 - 13. A_{nom}, B_{min-} (Normal A, Too low B)

Look carefully at the dot diagram on Page 14. The cross pattern has now grown longer arms because 4 outer dots have hopped completely outside the dashed boundary square lines.

B. The Generalized Robustness Formula

The Slide says: "It can be generalized that for n variables in a module, $6n + 1$ test cases can be designed with Robustness Testing method."

Since every single variable now has **6 testing points** instead of 4 (Min-, Min, Min+, Max-, Max, Max+), the calculation formula changes to:

Total Robustness Test Cases = $6n + 1$

- Example: For 2 variables ($n = 2$), $6(2) + 1 = 13$ test cases.

Quick Comparison Summary

Metric / Feature	Page 13: BVC (Normal)	Page 14: Robustness
Input Focus	Strictly Valid Values Only	Both Valid & Invalid Values
Points per Variable	4 (Min, Min+, Max-, Max)	6 (Min-, Min, Min+, Max-, Max, Max+)
Formula	$4n + 1$	$6n + 1$
Test Cases ($n = 2$)	9 total test cases	13 total test cases

2026-06-06 15:35:11 You Asked 

explain page 15 in details, in easy way and properly.

 Gemini 3.5 Flash

Based on the exact page sequence of your file, **Page 15** covers the **Worst-Case Testing Method**.

To understand this page easily, let's step back and look at the core philosophical shift that happens here. Up until now (in BVC and Robustness testing), we followed the *Single Fault Assumption* – assuming only one variable would hit a boundary at a time.

Page 15 throws that assumption out the window and introduces the **Multi-Fault Assumption**.

1. What is Worst-Case Testing?

The Slide says: "We can again extend the concept of BVC by assuming more than one variable on the boundary (Multi Fault Assumption). It is called the worst-case testing method."

- **The Core Idea:** In the real world, systems don't just fail when *one* thing goes wrong. Sometimes, a system crashes because **multiple inputs hit their extreme limits at the exact same moment**.
- **How it works:** Instead of keeping one variable at a normal value while testing the other, Worst-Case testing forces **both** variables to hit their boundaries simultaneously.

2. Visualizing the Dot Diagram on Page 15

If you look at the graph on Page 15, the pattern of dots changes completely:

- **BVC (Page 13)** looked like a **Cross (+)** because we only tested along the center axes.
- **Worst-Case (Page 15)** looks like a **Full Grid / Box**.

Because we allow variables to be at their extreme boundaries at the same time, we are finally testing the **corners** of our valid input domain. For example, we test what happens when Variable A is at its absolute minimum *and* Variable B is also at its absolute minimum.

3. Breaking Down the 25 Test Cases

The slide states that we take the original **9 test cases from BVC** and add **16 new combination cases**, bringing the total to **25 test cases**.

Every variable can now be tested at any of its 5 standard valid states:

{Min, Min+, Nominal, Max-, Max}. Let's look at how the new additions (cases 10 to 25) form the corners and edges of the grid:

- **The Bottom-Left Corner (Both variables are low):**
 - 10. A_{min}, B_{min}
 - 11. A_{min+}, B_{min}
 - 12. A_{min}, B_{min+}
 - 13. A_{min+}, B_{min+}
- **The Bottom-Right Corner (A is high, B is low):**
 - 14. A_{max}, B_{min}
 - 15. A_{max-}, B_{min}
 - 16. A_{max}, B_{min+} (Note: Slide typo says A_{maxo})
 - 17. A_{max-}, B_{min+}

- **The Top-Left Corner (A is low, B is high):**

- 18. A_{min}, B_{max}
- 19. A_{min+}, B_{max}
- 20. A_{min}, B_{max-}
- 21. A_{min+}, B_{max-}

- **The Top-Right Corner (Both variables are high):**

- 22. A_{max}, B_{max}
- 23. A_{max-}, B_{max}
- 24. A_{max}, B_{max-}
- 25. A_{max-}, B_{max-}

4. The Generalized Formula

The Slide says: "It can be generalized that for n variables in a module, 5^n test cases can be designed with Worst-Case Testing method."

Instead of a basic multiplication formula, Worst-Case testing grows **exponentially** because it evaluates every mathematical permutation.

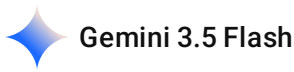
Total Worst-Case Test Cases = 5^n

- **Why 5?** Because each variable has exactly 5 valid test positions (Min, Min+, Nominal, Max-, Max).
- **Why to the power of n ?** Because you multiply those 5 possibilities for every new variable you add ($5 \times 5 \times 5 \dots$).

Formula Examples:

- If you have **1 variable** ($n = 1$): $5^1 = 5$ test cases (This matches BVC exactly).
- If you have **2 variables** ($n = 2$): $5^2 = 25$ test cases (The exact example on this slide).
- If you have **3 variables** ($n = 3$): $5^3 = 125$ test cases.

(Note: As you can see, Worst-Case testing generates a massive number of test cases very quickly as your software gets more complex!)



Based on the exact page sequence of your file, **Page 16** covers the **Robust Worst-Case Testing Method**.

This is the most aggressive, extreme, and comprehensive variation of Boundary Value Analysis (BVA). Let's break it down simply.

1. What is Robust Worst-Case Testing?

The Slide says: "We can again extend the concept of Robust by assuming more than one variable on the boundary (Multi Fault Assumption). It is called Robust worst-case testing method."

To understand this easily, look at how this method combines the two advanced techniques you studied earlier:

- From **Robustness Testing (Page 14)**, it takes the idea of testing **invalid (out-of-bounds) values** (Min- and Max+).
- From **Worst-Case Testing (Page 15)**, it takes the **Multi-Fault Assumption**, meaning it forces multiple variables to hit extreme limits at the exact same time.

In short: It tests what happens when multiple things go completely wrong at the same time, throwing both valid edge numbers and invalid/illegal numbers at the system simultaneously.

2. Visualizing the Dot Diagram on Page 16

Take a look at the graph on Page 16:

- **Worst-Case (Page 15)** formed a clean grid *inside* the square boundaries.
- **Robust Worst-Case (Page 16)** forms a massive, expanded grid that explodes **outside** the square boundary lines.

You now see clusters of 9 dots at every single corner of the diagram. This is because the test cases are checking variations like: Variable A is way too high (Max+) while Variable B is simultaneously way too low (Min-).

3. How many test cases does it make?

Every variable under this method can be in any of its **7 possible testing states**:
{Min-, Min, Min+, Nominal, Max-, Max, Max+}

Because we must test every possible combination of these 7 states for every variable, the slide gives us a generalized rule.

The Generalized Formula

The Slide says: "It can be generalized that for n variables in a module, 7^n test cases can be designed..."

(Note: There is a minor typo on your slide text which reads $7n$ instead of 7^n , but the title and context confirm it follows the exponential multi-fault progression).

Total Robust Worst-Case Test Cases = 7^n

- **Why 7?** Because each variable has exactly 7 valid/invalid test positions.
- **Why to the power of n ?** Because you multiply those 7 possibilities for every variable ($7 \times 7 \times 7 \dots$) to get every permutation.

Formula Examples:

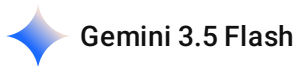
- If you have **1 variable** ($n = 1$): $7^1 = 7$ test cases.
- If you have **2 variables** ($n = 2$, like the graph on the slide): $7^2 = 49$ test cases.
- If you have **3 variables** ($n = 3$): $7^3 = 343$ test cases.

Master BVA Comparison Table

To wrap up the entire first half of your lecture, here is how all four BVA methods stack up against each other for a system with **2 variables** ($n = 2$):

Method Name	Focus	Formula	Total Tests ($n = 2$)	Diagram Pattern
Boundary Value Checking (BVC)	Valid values only, 1 variable at a time	$4n + 1$	9	Perfect Cross (+)
Robustness Testing	Valid & Invalid values, 1 variable at a time	$6n + 1$	13	Long-armed Cross
Worst-Case Testing	Valid values only, multiple variables at a time	5^n	25	Box / Grid
Robust Worst-Case Testing	Valid & Invalid values, multiple variables at a time	7^n	49	Huge Overlapping Grid

explain the examples from page 17 to 22 in details, in easy way and properly.



Here is the detailed, step-by-step explanation of the practical examples found on **Pages 17 to 22** of your lecture slides. These pages show exactly how to apply the boundary value formulas to real-world word problems.

EXAMPLE 1: The Prime Number Program

The Problem Definition (Page 17)

The Requirements: A program reads a single integer number within the range $[1, 100]$ and determines whether it is a prime number or not.

Here, we only have **one variable** ($n = 1$). The valid boundaries are:

- **Minimum (Min)** = 1
- **Maximum (Max)** = 100

A. Test Cases using BVC (Page 17)

Because $n = 1$, the total number of test cases required for Boundary Value Checking (BVC) is $4n + 1 = 4(1) + 1 = 5$.

Let's calculate the 5 points for our single variable:

1. **Min** = 1
2. **Min+** (Just above min) = 2
3. **Max** = 100
4. **Max-** (Just below max) = 99
5. **Nominal** (A standard average value in the middle) → The slide picks **53** (any nominal number between 50–55 works).

The BVC Test Case Table:

Test Case ID	Input Integer	Expected Output	Notes
1	1	Not a prime number	Absolute Minimum valid limit
2	2	Prime number	Just above minimum valid limit ($Min+$)
3	100	Not a prime number	Absolute Maximum valid limit
4	99	Not a prime number	Just below maximum valid limit ($Max-$)
5	53	Prime number	Average baseline input ($Nominal$)

B. Test Cases using Robustness Testing (Page 18)

Robustness testing checks what happens when things go out of bounds. The formula for a single variable is $6n + 1 = 6(1) + 1 = 7$ test cases.

To construct this, we copy the 5 valid test cases from BVC and add **2 invalid out-of-range boundaries**:

- **Min-** (One step below minimum limit): $1 - 1 = 0$
- **Max+** (One step above maximum limit): $100 + 1 = 101$

The Robustness Test Case Table:

Test Case ID	Input Integer	Expected Output	Notes
1	0	Invalid input	Out of bounds ($Min-$)
2	1	Not a prime number	Min boundary
3	2	Prime number	$Min+$ boundary
4	100	Not a prime number	Max boundary
5	99	Not a prime number	$Max-$ boundary
6	101	Invalid input	Out of bounds ($Max+$)
7	53	Prime number	Normal average item ($Nominal$)

C. Test Cases using Worst-Case Testing (Page 19)

The formula for Worst-Case testing is 5^n . Since we only have 1 variable here ($n = 1$), $5^1 = 5$ test cases.

Crucial Rule: When you only have **one** input variable, Worst-Case testing creates the exact same test cases as normal BVC. This is because multi-fault combinations are physically impossible when there is only one input to change.

EXAMPLE 2: The Power Function (a^b)

The Problem Definition (Page 20)

The Requirements: A program computes the mathematical power function a^b .

This problem has **two independent variables** ($n = 2$):

1. Variable **a** (Base): valid range is $[1, 10] \rightarrow \text{Min} = 1, \text{Max} = 10, \text{Nominal} = 5$
2. Variable **b** (Exponent): valid range is $[1, 5] \rightarrow \text{Min} = 1, \text{Max} = 5, \text{Nominal} = 3$

A. Test Cases using BVC (Page 20)

For 2 variables, BVC requires $4n + 1 = 4(2) + 1 = 9$ test cases. Because BVC uses the *Single Fault Assumption*, we keep one variable flat at its **nominal** value while shifting the other variable through its boundaries.

Shifting variable 'a' (while keeping $b = 3$):

- Test Case 1: a is at Min (1) $\rightarrow (1, 3)$
- Test Case 2: a is at $\text{Min} + (2)$ $\rightarrow (2, 3)$
- Test Case 3: a is at Max (10) $\rightarrow (10, 3)$
- Test Case 4: a is at $\text{Max} - (9)$ $\rightarrow (9, 3)$

Shifting variable 'b' (while keeping $a = 5$):

- Test Case 5: b is at Min (1) $\rightarrow (5, 1)$
- Test Case 6: b is at $\text{Min} + (2)$ $\rightarrow (5, 2)$
- Test Case 7: b is at $\text{Max} - (4)$ $\rightarrow (5, 4)$
- Test Case 8: b is at Max (5) $\rightarrow (5, 5)$

The Central Baseline:

- Test Case 9: Both variables are at their average baseline values $\rightarrow (5, 3)$

B. Test Cases using Robustness Testing (Page 21)

For 2 variables, Robustness requires $6n + 1 = 6(2) + 1 = 13$ test cases. We take our 9 valid test cases from BVC and inject **4 new test cases** to check what happens when either variable jumps completely outside the boundaries ($Min-$ or $Max+$).

- **Invalid low boundary for 'a' ($Min-$):** $a = 0$ (while $b = 3$) $\rightarrow (0, 3)$
- **Invalid high boundary for 'a' ($Max+$):** $a = 11$ (while $b = 3$) $\rightarrow (11, 3)$
- **Invalid low boundary for 'b' ($Min-$):** $b = 0$ (while $a = 5$) $\rightarrow (5, 0)$
- **Invalid high boundary for 'b' ($Max+$):** $b = 6$ (while $a = 5$) $\rightarrow (5, 6)$

Any test case containing one of these four configurations explicitly outputs an **"Invalid input"** error warning.

C. Test Cases using Worst-Case Testing (Pages 22 & 23)

Worst-case testing shifts away from holding one variable steady at its average value. It uses the multi-fault assumption, forcing us to try **every possible valid intersection** of the boundary positions.

The total number of test cases expands exponentially using the formula $5^n = 5^2 = 25$.

To write out all 25 test cases systematically without missing any, you can look at it as a grid. Each variable has a set of 5 safe checking values:

- Allowed values for a : $\{1, 2, 5, 9, 10\}$
- Allowed values for b : $\{1, 2, 3, 4, 5\}$

Page 23 maps out the complete combination array:

- Cases 1 to 9 are the standard cross-axes points copied from BVC.
- Cases 10 to 25 fill out the grid edges and outer corners where both extreme values clash together (e.g., Case 10 tests $a = 1, b = 1$, and Case 16 tests $a = 10, b = 5$).

This completely ensures that the calculation logic works securely even if the software hits its maximum processing thresholds on all inputs at the exact same moment.

2026-06-06 15:50:54

You Asked



explain page 24 in details, in easy way and properly.

Based on the exact page sequence of your file, **Page 24** transitions into the second major black-box testing technique: **Equivalence Class Testing** (also known as Equivalence Partitioning).

When the input domain of a program is too massive to test every single value, this technique provides a logical way to downsize your workload. Let's break down the slide's explanations in detail.

1. The Core Problem: Too Many Test Cases

The Slide says: "Typically the universe of all possible test cases is so large that you cannot try them all. You have to select a relatively small number of test cases to actually run. Which test cases should you choose?"

- **What it means:** Imagine a program that takes an employee's age as an integer between 18 and 65. If you wanted to test every *single possible integer*, that's 48 numbers. If the program accepts a 10-digit phone number, there are *billions* of possibilities.
 - Running every single possible input is called **Exhaustive Testing**, and it is practically impossible due to time, budget, and resource constraints. Therefore, testers need a smart strategy to select a representative subset.
-

2. The Solution: Equivalence Partitioning

The Slide says: "Partition the test cases into 'equivalence classes'. Each equivalence class contains a set of 'equivalent test cases'."

- **What it means:** To solve the problem of too many inputs, you divide (partition) the massive universe of test data into distinct groups or "buckets" called **equivalence classes**.
 - Every piece of data placed inside a specific bucket is considered "equivalent" to the others in that same bucket.
-

3. What makes inputs "Equivalent"?

The Slide says: "Two test cases are considered to be equivalent if we expect the program to process them both in the same way (i.e., follow the same path through the code)"

- **The Rule:** If you input two different numbers, and the software executes the exact same lines of code and treats them completely identically, those two numbers belong to the same equivalence class.
- **Easy Example:** Let's go back to our age program (valid range: 18 to 65).
 - If you test the number **25**, the program says "Access Granted."

- If you test the number **40**, the program does the exact same calculation and says "Access Granted."
- Because 25 and 40 follow the exact same logical path through the code, they are equivalent. Testing both is redundant.

4. The Core Assumption & The Benefit

The Slide says: "If you expect the program to process two test cases in the same way, only test one of them, thus reducing the number of test cases you have to run. With the assumption that if one test case in an equivalence class detects a bug, all other test cases in that class have the same probability of finding bugs."

This technique relies on a foundational testing assumption: **All elements in a class succeed or fail together.**

- **If it works:** If you pick a single representative number from a class (like 30) and the program processes it perfectly, you assume that *every other number* in that valid class (like 31, 32, 33...) will also work perfectly. You don't waste time running them.
- **If it fails:** If you pick a number from a class and it triggers a bug, you assume that any other number chosen from that same class will likely trigger the exact same bug.

Simple Summary of Page 24

Equivalence Partitioning is a "**divide and conquer**" strategy. Instead of testing hundreds of numbers blindly, you group similar inputs into logical categories. Then, you **pick just one representative value** from each category to run as a test case. If that one value passes, the whole group passes, saving massive amounts of development time and effort.

2026-06-07 14:32:00

You Asked



explain page 25 in details, in easy way and properly.

 Gemini 3.5 Flash

Based on the exact page sequence of your file, **Page 25** continues deeper into **Equivalence-Class Testing**, defining its formal definition and outlining its two ultimate goals. Let's break it down in a detailed, clear way.

1. The Formal Definition of Equivalence Partitioning

The Slide says: *"Equivalence partitioning is a method for deriving test cases wherein classes of input conditions called equivalence classes are identified such that each member of the class causes the same kind of processing and output to occur." *

- **In Easy Terms:** Think of a sorting machine at a fruit factory. It groups apples into buckets by size. The machine doesn't inspect every apple uniquely; it treats every apple inside the "Large Bucket" exactly the same way (processing) and packs them into a large crate (output).
- Similarly, in software testing, you group inputs together because you know the code will run them through the exact same logic block and deliver the same type of result.

2. Goal #1: Completeness

The Slide says: *"Completeness: Without executing all the test cases, we strive to touch the completeness of testing domain." *

- **The Problem:** As we learned on page 24, you cannot test everything because the input domain is too large.
- **The Solution: Completeness** means making sure your test cases cover *every possible scenario* or type of input the program might encounter. By creating equivalence classes for every type of acceptable and unacceptable input, you can confidently say you have tested the "complete" functionality of the program by running just a tiny fraction of total data points. You don't leave any stone unturned.

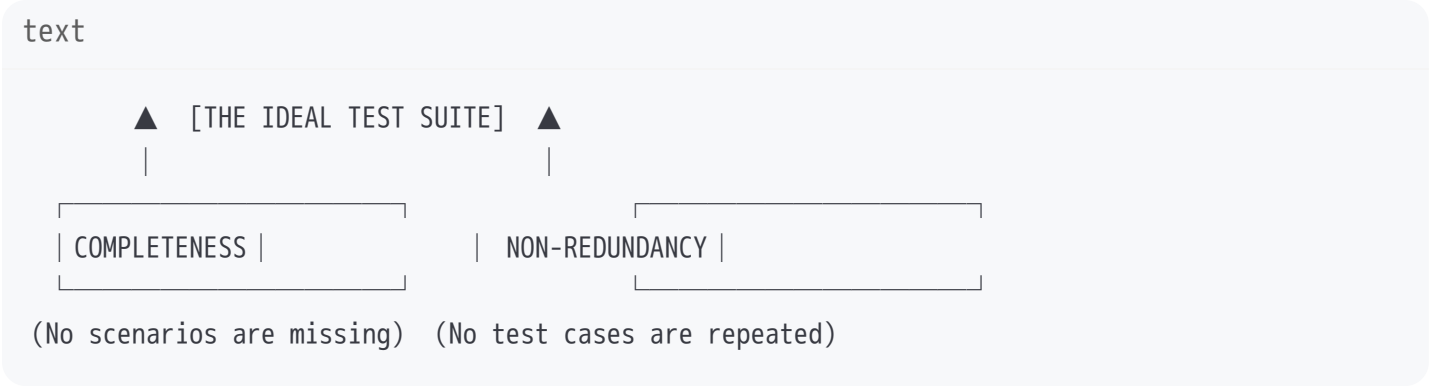
3. Goal #2: Non-Redundancy

The Slide says: *"Non-redundancy: When the test cases are executed having inputs, then there is redundancy in executing the test cases. Time and resources are wasted in executing these redundant test cases, as they explore the same type of bug." *

- **The Problem:** Redundancy means doing the exact same work over and over again for no good reason.
- **Easy Example:** If a text box is supposed to accept any positive number, and you write 50 different test cases using the numbers 5, 6, 7, 8... up to 54, you are being highly redundant.
- **The Solution:** Why? Because if the program has a bug that breaks when processing the number 5, it will almost certainly break exactly the same way for the number 6. Testing both numbers wastes your valuable **time and resources**. Equivalence class testing strictly enforces **non-redundancy** by making sure you select only *one* representative input from each bucket, eliminating useless, repetitive testing.

Visualizing the Ideals of Page 25

A perfect test suite balances these two forces like a see-saw:



Summary of Page 25

Page 25 explains that Equivalence Partitioning is designed to optimize your testing suite perfectly. It gives you two absolute guardrails:

- 1. **Completeness:** Ensure you have tested at least one value from every functional scenario so bugs have nowhere left to hide.
- 2. **Non-redundancy:** Ensure you aren't testing values that do the exact same thing, preventing a massive waste of testing time, effort, and processing power.

2026-06-07 14:33:10 You Asked 

explain page 26,27,28 in details, in easy way and properly.

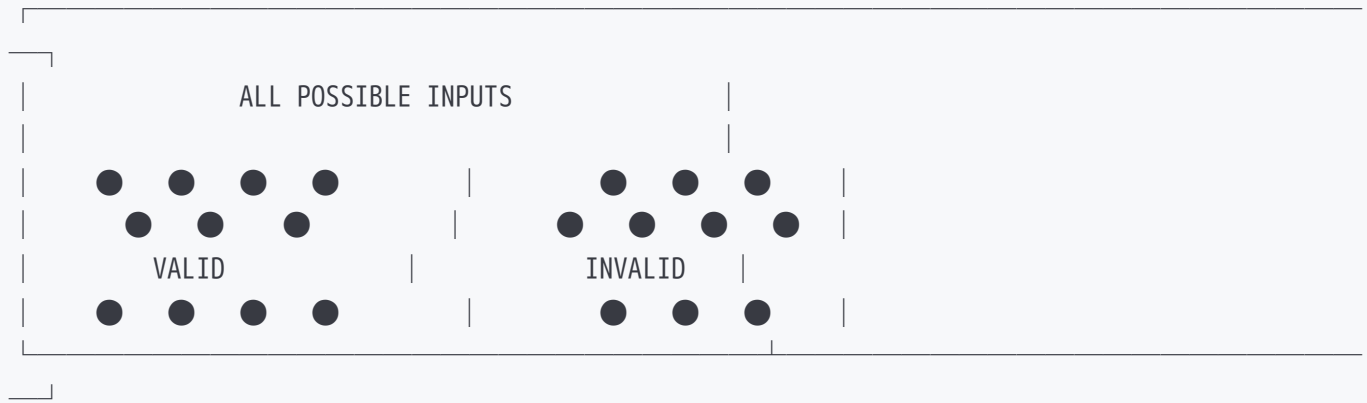
 Gemini 3.5 Flash

Here is the detailed, easy-to-understand breakdown of **Pages 26, 27, and 28** from your lecture slides, which use clear circle diagrams to show how testers physically partition input data.

Page 26: First-Level Partitioning (Valid vs. Invalid)

Page 26 introduces the very first step a tester takes when looking at a massive universe of potential test data.

text



A. The Big Picture Split

The Slide says: "First-level partitioning: Valid vs. Invalid test cases"

Imagine the large outer circle on this slide represents every single value a user could possibly type into a field. To make sense of it, you draw a single line right down the middle, splitting the universe into two basic halves:

1. **The Valid Side:** Data that the program *expects* to receive.
2. **The Invalid Side:** Data that is completely illegal or unexpected.

B. Breaking Down the Halves

- **Valid Equivalence Classes:** These are clean, normal inputs that follow the software's rules. When you use these, the software should perform its standard operations successfully.
- **Invalid Equivalence Classes:** These are incorrect inputs. Why test them? Because we must verify that the program doesn't crash when a user makes a mistake. Instead, it should handle the invalid input safely by showing an error message like *"Invalid input"* or *"Please try again"*.

Page 27: Creating Sub-Classes (The Heuristic Process)

Page 27 moves to the next logical stage. Just splitting data into "Valid" and "Invalid" isn't enough; we need to subdivide those two halves into smaller, specific categories.

A. Grouping into Specific Buckets

As you can see in the diagram on Page 27, inside the valid and invalid zones, the dots are now enclosed within smaller, irregular shapes. These are your specific **Equivalence Classes**.

- For example, if a program accepts scores from 0 to 100 to give letter grades, the "Valid" side will be split into multiple sub-classes: one bucket for A-grade scores (90-100), one for B-grade scores (80-89), and so on.

- The "Invalid" side will also get specific buckets: one for negative numbers (less than 0) and one for massive numbers (greater than 100).

B. The Heuristic Rule

The Slide says: *"There are no well-defined rules for identifying equivalence classes, as it is a heuristic process."*

- **What it means:** The word **heuristic** means "learning or discovering by trial-and-error, experience, and practical guesswork." There is no magic mathematical formula that instantly builds these classes for you. A tester must use their own brain, look at the system requirements, and logically deduce the best way to group the data based on how they expect the code to act.

Page 28: Selecting the Final Test Cases

Page 28 brings everything together by showing you exactly how to pick your final test cases from the groups you drew on Page 27.

In the diagram on Page 28, you will notice that every single sub-bucket now has **one dot circled in red**. Those red circles are the chosen representative values that will become your actual test cases.

A. The Selection Guidelines

The slide gives two strict instructions on how to pick these values:

- **Guideline 1: Valid Class Multi-Tasking**

"Assign a unique identification number to each equivalence class. Write a new test case covering as many of the uncovered valid equivalence classes as possible, until all valid equivalence classes have been covered..."

- **In Plain English:** To save time, you can combine multiple *valid* inputs into a single test case. If you have three different valid fields on a form, you can test a valid value for Class #1, Class #2, and Class #3 all at the same time in one single test run.

- **Guideline 2: Invalid Class Isolation (CRITICAL RULE)**

"Write a test case that covers one, and only one, of the uncovered invalid equivalence classes, until all invalid equivalence classes have been covered..."

- **In Plain English:** You must **never** mix multiple invalid inputs into a single test case. You must test them one at a time.
- **Why?** Because if you fill out a form with an invalid username *and* an invalid password at the same time, the system will reject it immediately on the username. You will never know if the password validation logic actually works or if it has a hidden bug! Testing invalid classes strictly **one by one** ensures every error-handling routine is verified perfectly.

Summary Checklist for Pages 26–28

1. **Page 26:** Split your input domain into Valid territory and Invalid territory.
2. **Page 27:** Break those territories down into smaller logical sub-groups using your experience.
3. **Page 28:** Pick exactly one representative value from each group. You can bundle valid tests together, but you must test invalid tests strictly on their own.

2026-06-07 14:48:17

You Asked 

explain page 30 in details, in easy way and properly.

 Gemini 3.5 Flash

Based on the exact page sequence of your file, **Page 30** sets up the mathematical foundation and notation used for the four sub-techniques of Equivalence Class Testing.

Before you can write test cases using methods like *Weak Normal* or *Strong Robust* testing, you have to map out your variables using intervals. Page 30 teaches you exactly how to read and build this mathematical map.

1. Defining the Function and Variables

The Slide says: "Function F is implemented and a function F , of two variables x_1 and x_2 ."

- **In Easy Terms:** Imagine you are testing a feature (Function F) that takes exactly two inputs: x_1 and x_2 . For example, x_1 could be a user's birth day (1–31) and x_2 could be their birth month (1–12).

2. Understanding Closed vs. Open Intervals

At the bottom of the page, there is a crucial mathematical remark:

The Slide says: "Remarks: $[$ = closed interval, $($ = open interval"

This notation tells you whether the boundary number itself is included in the test bucket:

- **Closed Interval** $[$ or $]$: Means the number **is included** ("equal to or greater/less than").
- **Open Interval** $($ or $)$: Means the number **is excluded** ("strictly less than" or "strictly greater than").

3. Breaking Down the Valid Intervals

The slide sets up boundaries for both variables using these intervals:

Variable x_1 Boundaries

The Slide says: " $a \leq x_1 \leq d$ with intervals $[a,b)$, $[b,c)$, $[c,d]$ "

This means x_1 is valid anywhere between absolute minimum a and absolute maximum d . It splits this valid territory into three distinct valid buckets (equivalence classes):

1. **First bucket $[a,b)$** : Includes a , up to but excluding b ($a \leq x_1 < b$).
2. **Second bucket $[b,c)$** : Includes b , up to but excluding c ($b \leq x_1 < c$).
3. **Third bucket $[c,d]$** : Includes both c and d ($c \leq x_1 \leq d$).

Variable x_2 Boundaries

The Slide says: " $e \leq x_2 \leq g$ with intervals $[e,f)$, $[f,g]$ "

Similarly, x_2 is valid between absolute minimum e and absolute maximum g . It splits this into two valid buckets:

1. **First bucket $[e,f)$** : Includes e , up to but excluding f ($e \leq x_2 < f$).
2. **Second bucket $[f,g]$** : Includes both f and g ($f \leq x_2 \leq g$).

4. Identifying the Invalid Territory

The Slide says: "So, invalid value for x_1 and x_2 as follows, $x_1 < a$ and $x_1 > d$, $x_2 < e$ and $x_2 > g$ "

Now that we know exactly where the valid territory starts and ends, anything outside of it is automatically an **invalid equivalence class**.

- For x_1 , falling below the absolute minimum ($x_1 < a$) or shooting past the absolute maximum ($x_1 > d$) is invalid.
- For x_2 , falling below the absolute minimum ($x_2 < e$) or shooting past the absolute maximum ($x_2 > g$) is invalid.

Visualizing Page 30's Map

If you look at the entire input domain as a geographic map, Page 30 sets up a grid of **Valid States** surrounded by an ocean of **Invalid States**:

text

Invalid Territory ($x_2 > g$)

Invalid Territory (x1 < a)		[Class 4]		[Class 5]		Invalid Territory (x1 > d)
		[e, f)		[f, g]		
	----- Territory					
		[Class 1]		[Class 2]		
		[a, b)		[b, c)		

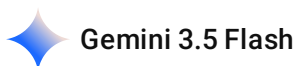
Invalid Territory ($x_2 < e$)

Summary of Page 30

Page 30 doesn't give you test cases yet. It simply sets up the rules of the game. It tells you that x_1 has **3 valid sub-classes**, x_2 has **2 valid sub-classes**, and both have **2 invalid zones** (too low or too high). The following pages use this exact setup to create testing grids.

2026-06-07 15:06:49 You Asked 

explain page 31 in details, in easy way and properly.



Based on the exact page sequence of your file, **Page 31** takes the mathematical definitions we established on page 30 and builds the very first sub-technique of Equivalence Class Testing: **Weak Normal Testing (WN)**.

Let's look at how this method creates specific valid test cases using a visual diagram and simple logic.

1. Defining the Valid Equivalence Classes (Valid EC)

First, the slide identifies all the **valid** buckets or classes (Ec) based on the boundary intervals from the previous page:

- **For Variable x_1 (3 Valid Classes):**
 - $Ec1 = \{x_1 : a \leq x_1 < b\}$

- $Ec2 = \{x1 : b \leq x1 < c\}$
- $Ec3 = \{x1 : c \leq x1 \leq d\}$
- **For Variable $x2$ (2 Valid Classes):**
 - $Ec4 = \{x2 : e \leq x2 < f\}$
 - $Ec5 = \{x2 : f \leq x2 \leq g\}$

This means there are a total of **5 valid classes** that our testing team needs to check.

2. Understanding the "Weak" and "Normal" Rules

To understand this easily, let's look at why it's named **Weak Normal**:

- **The "Normal" Part:** This means we are only concerned with **valid** inputs. We completely ignore the invalid territories (the out-of-bounds inputs) on this page.
- **The "Weak" Part (Single Fault Assumption):** > **The Slide says:** *"Weak normal equivalence class testing is also known as single fault assumption."*

This is identical to the philosophy we used in basic boundary testing. We operate under the assumption that bugs are usually caused by a single thing going wrong at a time, rather than a combination of multiple things failing simultaneously.

3. Visualizing the Diagram on Page 31

The slide features a shaded grey rectangle that visualizes the valid territory across the $x1$ and $x2$ axes.

Because $x1$ has 3 intervals and $x2$ has 2 intervals, the valid territory is divided into a grid of **6 distinct intersection boxes** ($3 \times 2 = 6$).

Look closely at the **black dots** inside the grey grid on your slide. There are exactly **3 dots**. Why 3 dots instead of 6?

The Core Selection Rule:

The Slide says: *"One variable from each equivalence class is tested by the team."*

The rule of Weak Normal testing is to cover every single identified class ($Ec1$ through $Ec5$) at least once, using the **minimum number of test cases possible**.

- Instead of running a test for all 6 intersection boxes, we can strategically place our dots so that one test point checks multiple classes at the same time!

Let's look at how the 3 dots on the slide achieve this:

1. **Bottom-Left Dot:** Placed in the box where $Ec1$ and $Ec4$ intersect. This single test successfully covers both $Ec1$ and $Ec4$ at the same time.
2. **Top-Middle Dot:** Placed in the box where $Ec2$ and $Ec5$ intersect. This covers both $Ec2$ and $Ec5$.
3. **Bottom-Right Dot:** Placed in the box where $Ec3$ and $Ec4$ intersect. This covers $Ec3$ (and repeats $Ec4$).

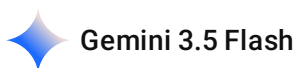
By the time you run these **3 test cases**, you can check off all 5 valid equivalence classes successfully.

Summary of Weak Normal Testing (Page 31)

- It focuses strictly on **valid** range groups.
- It follows the **Single Fault Assumption**.
- The goal is to be highly efficient: pick just enough test points so that **every valid class is covered at least once**, minimizing redundancy and saving execution time.

2026-06-07 15:16:57 You Asked 

explain the example in page 33



Based on the exact slide sequence of the document, **Page 33** provides a concrete numerical example of **Weak Normal (WN) Equivalence Class Testing** using an addition program. Let's break it down in a detailed, clear way.

1. Setting Up the Ranges and Classes (Valid EC)

We are testing a program that takes two input numbers, $x1$ and $x2$, and adds them together. The slide maps out specific valid ranges (intervals) for both numbers:

- **For Variable $x1$:** The total valid range is from 5 to 20. This range is split into **3 valid classes**:
 - $Ec1 = \{x1 : 5 \leq x1 < 10\}$ (Numbers from 5 up to 9)
 - $Ec2 = \{x1 : 10 \leq x1 < 15\}$ (Numbers from 10 up to 14)
 - $Ec3 = \{x1 : 15 \leq x1 \leq 20\}$ (Numbers from 15 up to 20)
- **For Variable $x2$:** The total valid range is also from 5 to 20. It is split into **2 valid classes**:

- $Ec4 = \{x2 : 5 \leq x2 < 10\}$ (Numbers from 5 up to 9)
 - $Ec5 = \{x2 : 10 \leq x2 \leq 20\}$ (Numbers from 10 up to 20)
-

2. Visualizing the Grid Pattern

If you look at the grid diagram on Page 33, the intersections of these intervals form $3 \times 2 = 6$ potential boxes inside the shaded blue valid area.

Because this is **Weak Normal** testing, we follow the **Single Fault Assumption**. Our goal is to test at least one representative value from each of our 5 classes ($Ec1$ through $Ec5$) using the *minimum number of test cases possible*. Instead of running 6 tests, we can cover everything using just **3 strategic test points** (represented by the black dots on the grid).

3. Breaking Down the 3 Test Cases

The table at the bottom of page 33 lists the final test cases selected to cover all classes efficiently:

Test Case WN1

- **Classes Covered:** $Ec1$ and $Ec4$
- **Inputs Chosen:** $x1 = 5$ (which fits perfectly inside $Ec1$) and $x2 = 6$ (which fits inside $Ec4$)
- **Expected Result:** $5 + 6 = 11$
- **Achievement:** This single test run checks off both $Ec1$ and $Ec4$ simultaneously.

Test Case WN2

- **Classes Covered:** $Ec2$ and $Ec5$
- **Inputs Chosen:** $x1 = 12$ (fits inside $Ec2$) and $x2 = 15$ (fits inside $Ec5$)
- **Expected Result:** $12 + 15 = 27$
- **Achievement:** This test point validates $Ec2$ and $Ec5$ together.

Test Case WN3

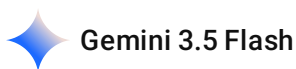
- **Classes Covered:** $Ec3$ and $Ec4$
 - **Inputs Chosen:** $x1 = 17$ (fits inside $Ec3$) and $x2 = 7$ (fits inside $Ec4$)
 - **Expected Result:** $17 + 7 = 24$
 - **Achievement:** This test point validates the final remaining class, $Ec3$ (while safely repeating $Ec4$).
-

Summary of Page 33

By executing just these **3 test cases**, the testing team has successfully passed a representative number through every single valid category of the program ($Ec1, Ec2, Ec3, Ec4, Ec5$). It proves the efficiency of Weak Normal testing: maximum class coverage with minimum execution redundancy.

2026-06-07 15:19:42 You Asked 

explain page 34 aND 35 in details, in easy way and properly.



Here is the detailed, easy-to-understand breakdown of **Page 34** and **Page 35** from your slides, which cover **Strong Normal Testing**.

Page 34: 2. Strong Normal Testing (SN)

Page 34 introduces the second sub-technique of Equivalence Class Testing: **Strong Normal Testing (SN)**. Let's look at how this method elevates your testing coverage compared to Weak Normal testing.

A. The Core Rule: Cartesian Product

The Slide says: "Test cases taken from each element of Cartesian product of the equivalence classes. Cartesian product guarantees notion of completeness."

- **What it means in simple terms:** Weak Normal testing (Page 31) was lazy—it just wanted to make sure every class was touched at least once. **Strong Normal testing is thorough.** It demands that you test **every possible combination** of your valid equivalence classes.
- Mathematically, it multiplies your classes together (a Cartesian product). If Variable $x1$ has 3 valid classes and Variable $x2$ has 2 valid classes, Strong Normal testing forces you to test $3 \times 2 = 6$ combinations.

B. The "Multiple Fault Assumption"

The Slide says: "SN is a 'multiple fault assumption'"

- **What it means:** Weak Normal assumed only one thing would go wrong at a time (Single Fault). Strong Normal operates under the **Multiple Fault Assumption**. It believes that a program might work fine when $x1$ is in its first bucket and $x2$ is in its first bucket, but it might break specifically when $x1$ is in its third bucket *and* $x2$ is in its second bucket simultaneously.

C. Visualizing the Grid on Page 34

Look at the shaded grey rectangle on Page 34:

- Unlike Page 31 (which only had 3 dots scattered on the grid) , the diagram on Page 34 has **black dots in every single intersection box**.
 - There are exactly **6 dots** because every valid row must meet every valid column.
-

Page 35: Example of Strong-Normal Testing

Page 35 applies these thorough rules to the exact same addition program we saw in the Weak Normal example on Page 33.

A. Re-stating the Valid Classes

The classes remain exactly the same:

- **Variable $x1$ (3 Classes):** $Ec1$ [5, 10), $Ec2$ [10, 15), $Ec3$ [15, 20]
- **Variable $x2$ (2 Classes):** $Ec4$ [5, 10), $Ec5$ [10, 20]

B. Breaking Down the 6 Test Cases

Because we must test the full Cartesian product ($3 \times 2 = 6$), the table at the bottom of Page 35 outlines **6 mandatory test cases**:

1. **SN1 (EC1, EC4):** Tests the intersection of the first buckets.
 - Inputs: $x1 = 5, x2 = 6 \rightarrow$ Expected Result: **11**
 2. **SN2 (EC1, EC5):** Combines the first bucket of $x1$ with the second bucket of $x2$.
 - Inputs: $x1 = 7, x2 = 12 \rightarrow$ Expected Result: **19**
 3. **SN3 (EC2, EC4):** Combines the second bucket of $x1$ with the first bucket of $x2$.
 - Inputs: $x1 = 10, x2 = 5 \rightarrow$ Expected Result: **15**
 4. **SN4 (EC2, EC5):** Combines the second bucket of $x1$ with the second bucket of $x2$.
 - Inputs: $x1 = 12, x2 = 15 \rightarrow$ Expected Result: **27**
 5. **SN5 (EC3, EC4):** Combines the third bucket of $x1$ with the first bucket of $x2$.
 - Inputs: $x1 = 17, x2 = 7 \rightarrow$ Expected Result: **24**
 6. **SN6 (EC3, EC5):** Combines the third bucket of $x1$ with the second bucket of $x2$.
 - Inputs: $x1 = 15, x2 = 15 \rightarrow$ Expected Result: **30**
-

Quick Review: Weak Normal vs. Strong Normal

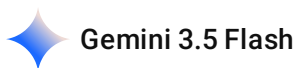
To wrap up these two pages, compare them to see the difference clearly:

- **Weak Normal (Page 33):** Used only **3 test cases** because it just wanted to check off each bucket name at least once. It was fast but left some combination gaps.
- **Strong Normal (Page 35):** Uses **6 test cases** because it maps out every single valid square yard of the grid intersection. It takes more time but provides complete valid combination coverage.

2026-06-07 15:33:07

You Asked 

explain page 36 aND 37 in details, in easy way and properly.



Here is the detailed, easy-to-understand breakdown of **Page 36** and **Page 37** from your slides, which transition into **Weak Robust Testing**.

Page 36: 3. Weak Robust Testing (WR)

Up until now, you have only dealt with **Normal** testing techniques, which strictly stay inside the safe, valid zones. Page 36 introduces the **Robust** dimension. In software testing, "Robust" means the program's ability to handle bad, illegal, or out-of-range data gracefully without crashing or exploding.

Let's look at the core rules of this method:

A. Introducing Invalid Equivalence Classes (Invalid EC)

The Slide says: "Additional consider in Invalid EC: $Ec6 = \{x1 : x1 < a\}$, $Ec7 = \{x1 : x1 > d\}$, $Ec8 = \{x2 : x2 < e\}$, $Ec9 = \{x2 : x2 > g\}$ "

To perform robust testing, we take our existing valid map and formally identify the illegal zones outside the boundaries:

- **For Variable $x1$:** Falling below the minimum value a ($Ec6$) or shooting past the maximum value d ($Ec7$).
- **For Variable $x2$:** Falling below the minimum value e ($Ec8$) or shooting past the maximum value g ($Ec9$).

B. The "Weak" Rule + "Single Fault Assumption"

The Slide says: "Invalid inputs - each test case has one invalid value, single fault should cause failure as 'single fault assumption'."

Just like before, the word **Weak** tells us we are strictly following the **Single Fault Assumption**.

- **The Rule:** When testing the error-handling of the system, you must input **exactly one invalid value at a time** while keeping all other variables perfectly normal and valid.
- *Why?* If you type an invalid $x1$ and an invalid $x2$ at the same time, the program will throw an error message and reject the input. However, you won't know which invalid variable triggered the message, leaving a potential blind spot in your validation logic.

C. Visualizing the Diagram on Page 36

Look at the grid diagram on Page 36:

- You see the same 3 black dots inside the central grey rectangle—these are the original valid test cases from Weak Normal testing.
- Surrounding the grey box, you now see **4 red dots**. Each red dot represents a test point placed precisely in an invalid zone ($Ec6$, $Ec7$, $Ec8$, $Ec9$). Notice that they never occupy the corners; they stay centered along the outer rows and columns to fulfill the single-fault rule.

Page 37: Example of Weak-Robust Testing

Page 37 applies these robust principles back to our addition program example.

A. Mapping the Entire Universe of Classes

We now have a total of **9 equivalence classes** to keep track of:

- **Valid Classes:** $Ec1$, $Ec2$, $Ec3$ (for $x1$) and $Ec4$, $Ec5$ (for $x2$).
- **Invalid Classes:**
 - * $Ec6 = \{x1 : x1 < 5\}$ (Too low $x1$)
 - $Ec7 = \{x1 : x1 > 15\}$ (Too high $x1$) (*Note: Based on previous slides, the absolute max of $x1$ is 20, but the example slide explicitly sets the testing partition threshold here at 15.*)
 - $Ec8 = \{x2 : x2 < 10\}$ (Too low $x2$)
 - $Ec9 = \{x2 : x2 > 20\}$ (Too high $x2$)

B. Breaking Down the 7 Test Cases

The table outlines **7 distinct test cases**:

The Valid Core (Copied from Weak Normal):

- **WR1 (EC1, EC4):** Inputs $x1 = 5$, $x2 = 6$. Both are valid. → *Expected Result: 11*
- **WR2 (EC2, EC5):** Inputs $x1 = 12$, $x2 = 15$. Both are valid. → *Expected Result: 27*
- **WR3 (EC3, EC4):** Inputs $x1 = 17$, $x2 = 7$. Both are valid. → *Expected Result: 24*

The Robust Outer Ring (Testing Invalid Buckets One-by-One):

- **WR4 (EC6, EC4):** Tests the too-low exception for $x1$.

- Inputs: $x1 = 4$ (Invalid!) and $x2 = 8$ (Valid)
- Expected Result: "X1 is out of range"
- **WR5 (EC7, EC4):** Tests the too-high exception for $x1$.
 - Inputs: $x1 = 23$ (Invalid!) and $x2 = 8$ (Valid)
 - Expected Result: "X1 is out of range"
- **WR6 (EC3, EC8):** Tests the too-low exception for $x2$.
 - Inputs: $x1 = 18$ (Valid) and $x2 = 4$ (Invalid!)
 - Expected Result: "X2 is out of range"
- **WR7 (EC3, EC9):** Tests the too-high exception for $x2$.
 - Inputs: $x1 = 18$ (Valid) and $x2 = 23$ (Invalid!)
 - Expected Result: "X2 is out of range"

Summary of Weak Robust Testing

Weak Robust testing gives you a beautifully optimized balance. It ensures that:

1. Every standard valid category works seamlessly together.
2. The program safely catches errors and throws clean warning displays if a user types a bad value into *any* input field.

2026-06-07 15:35:39

You Asked 

explain page 38 in details, in easy way and properly.

 Gemini 3.5 Flash

Based on the exact page sequence of your file, **Page 38** covers the fourth and final sub-technique of Equivalence Class Testing: **Strong Robust Testing (SR)**.

This is the most aggressive and absolute combination strategy in partitioning testing. Let's break it down into simple, easy-to-understand rules.

1. What is Strong Robust Testing?

To understand this method easily, look at how its name combines two ideas you already know:

- **Robust:** This means we must look at both **valid** and **invalid** (out-of-bounds) equivalence classes.
- **Strong:** This means we throw out the "Single Fault Assumption" and embrace the **Multiple Fault Assumption**. We assume that multiple things can go wrong simultaneously.

In short: Strong Robust testing checks what happens when multiple inputs are completely invalid at the exact same time.

2. Visualizing the Dot Diagram on Page 38

Look closely at the grid diagram on Page 38:

- Inside the central grey rectangle, you can see the standard **black dots** representing valid combinations.
- Surrounding the grey box, you see an explosion of **red dots**.

Unlike Weak Robust testing (where red dots were only allowed in the straight rows and columns), Strong Robust testing places red dots everywhere—including the **outer corners**. These outer corner dots represent test cases where *both* Variable $x1$ and Variable $x2$ are broken or invalid at the same time.

3. The Rule of the Cartesian Product

The Slide says: "Test cases taken from each element of Cartesian product of the Valid EC and Invalid EC"

To calculate exactly how many test cases this method generates, you multiply the *total* number of classes (both valid and invalid) for each variable together.

Let's look at the math for our ongoing addition function example:

- **Variable $x1$ has 5 total classes:** 3 Valid classes ($Ec1, Ec2, Ec3$) + 2 Invalid classes ($Ec6, Ec7$).
- **Variable $x2$ has 4 total classes:** 2 Valid classes ($Ec4, Ec5$) + 2 Invalid classes ($Ec8, Ec9$).

To get the complete Cartesian product, you multiply them together:

Total Test Cases = $5 \times 4 = 20$ test cases

If you count every single dot (both black and red) on the Page 38 diagram, you will find exactly 20 dots.

4. What do these test cases look like?

While your slide shows the visual diagram, if we were to write out the table for these test cases, it would look like this:

1. **The 6 Normal Cases:** Valid $x1$ + Valid $x2$ (e.g., $x1 = 5, x2 = 6$).
2. **The Single-Fault Invalid Cases:** One variable is invalid, the other is valid (e.g., $x1 = 4$ [Invalid], $x2 = 8$ [Valid]).
3. **The Multi-Fault Invalid Cases (The Corners):** Both variables are invalid at the same time! For example, you test what happens when a user types an $x1$ that is too low **and** an $x2$ that is too high simultaneously (e.g., $x1 = 4$ [Invalid] and $x2 = 23$ [Invalid]).

Summary of the 4 Equivalence Class Methods

Now that you have finished all 4 sub-techniques, here is the ultimate cheat sheet to remember how they differ based on **Page 29 to 38**:

Technique Name	Focus	Core Assu mption	How to pick test points?
1. Weak Normal (Page 31)	Valid inputs o nly.	Single Faul t.	Just enough points to touch each class name once.
2. Strong Normal (Page 34)	Valid inputs o nly.	Multiple Fa ult.	Cartesian product of all valid boxes.
3. Weak Robust (Page 36)	Valid + Invalid inputs.	Single Faul t.	Test invalid inputs strictly one at a time.
4. Strong Robust (Page 38)	Valid + Invalid inputs.	Multiple Fa ult.	Cartesian product of absolutely everythi ng (including corners).

2026-06-07 15:37:17

You Asked 

explain page 39 aND 40 in details, in easy way and properly.

 Gemini 3.5 Flash

Here is the detailed, easy-to-understand breakdown of **Page 39** and **Page 40** from your slides, which introduce a classic software testing scenario: **The Next Date Problem**.

Page 39: The Next Date Problem (Problem Definition)

Page 39 defines the problem requirements. This is a famous baseline example used in Computer Science courses to show how tricky input conditions can get.

A. What is the program supposed to do?

The Slide says: "A program determines the next date in the calendar. Its input is entered in the form of `<ddmmyyyy>`... Its output would be the next date or an error message 'invalid date.'"

- **In Plain English:** You type a specific date into the software (Day, Month, Year). The program calculates what tomorrow's date would be and prints it out. If you enter a date that doesn't exist, it displays an error message.
- **Example 1:** Input `15 06 1912` → Output `6/16/1912`.
- **Example 2:** Input `32 06 1912` → Output `invalid date`.

B. Understanding the General Range Restrictions

The slide lays down the standard mathematical range boundaries for the inputs:

- **Month (mm):** Must be between 1 and 12 ($1 \leq mm \leq 12$).
- **Day (dd):** Must be between 1 and 31 ($1 \leq dd \leq 31$).
- **Year ($yyyy$):** Must be between 1900 and 2025 ($1900 \leq yyyy \leq 2025$).

Page 40: Next Date Problem: Basic Equivalence Classes

Page 40 takes those general boundaries from Page 39 and performs a **First-Level Partitioning** (splitting everything into Valid and Invalid buckets, just like you saw on Page 26).

Let's look at the generated classes:

1. Valid Equivalence Classes (Valid EC)

These represent standard data parameters that fall safely inside our specified rule ranges:

- **M1 (Valid Month):** Any month number from 1 to 12 ($1 \leq \text{month} \leq 12$).
- **D1 (Valid Day):** Any day number from 1 to 31 ($1 \leq \text{day} \leq 31$).
- **Y1 (Valid Year):** Any year number from 1900 to 2025 ($1900 \leq \text{year} \leq 2025$).

2. Invalid Equivalence Classes (Invalid EC)

These represent bad inputs that fall outside the limits and should trigger an "invalid date" error. Each variable has **two invalid zones** (either too low or too high):

- **For Month:**
 - **M2 (Too Low):** Any month less than 1 ($\text{month} < 1$).
 - **M3 (Too High):** Any month greater than 12 ($\text{month} > 12$).

- **For Day:**
 - **D2 (Too Low):** Any day less than 1 ($\text{day} < 1$).
 - **D3 (Too High):** Any day greater than 31 ($\text{day} > 31$).
- **For Year:**
 - **Y2 (Too Low):** Any year less than 1900 ($\text{year} < 1900$).
 - **Y3 (Too High):** Any year greater than 2025 ($\text{year} > 2025$).

The Hidden Structural Problem with Page 40

While Page 40 gives a clean first-level look, it actually contains a major logical trap.

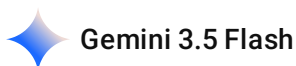
- **The Problem:** Months have different lengths!
- If you pick a day value of 31 (which matches the "Valid Day" class $D1$) and pair it with month 4 (April, which matches the "Valid Month" class $M1$), the input becomes **April 31st**.
- April 31st is an impossible date! But according to the simplistic classes on Page 40, both numbers are technically "valid."

Because these basic partitions fail to handle complex reality, your slides will reveal an **Improved Input Equivalence Classes** structure later on Page 44 to handle variations like 30-day months, 31-day months, and February leap years safely.

2026-06-07 16:04:14

You Asked 

complete page 41



Based on the exact sequence of your document, **Page 41** shows how the basic valid classes from Page 40 are translated into final test cases for both **Weak Normal** and **Strong Normal** testing.

The slide provides the starting format of a test table but leaves it as a single row to demonstrate how simple the setup is when using a basic partitioning strategy. Let's complete the page and fill out the full testing logic properly.

1. Recapping the Rules for Page 41

From Page 40, we established that we have exactly **3 Valid Equivalence Classes**:

- *M1*: Month between 1 and 12
- *D1*: Day between 1 and 31
- *Y1*: Year between 1900 and 2025

Why do Weak Normal (WN) and Strong Normal (SN) look the same here?

- **Weak Normal Rule:** Touch each valid class name (*M1*, *D1*, *Y1*) at least once using the minimum number of test cases possible.
- **Strong Normal Rule:** Test every possible combination of valid classes (Cartesian Product).
- **The Result:** Because each of our variables only has **one single valid class** defined on Page 40 ($1 \times 1 \times 1 = 1$), both the Weak Normal and Strong Normal methods yield the exact same **1 test case**.

2. The Completed Page 41 Test Table

To complete Page 41 properly, we select standard representative numbers that fall cleanly inside our valid *M1*, *D1*, and *Y1* classes. The slide picks June 15, 1912:

Test Case	Month	Day	Year	Expected Results	Covered Classes
WN1, SN1	6	15	1912	6/16/1912	Month $\in M1$, Day $\in D1$, Year $\in Y1$

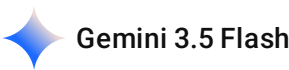
Breakdown of the Test Case:

- **Month = 6:** Valid because June falls within $[1, 12]$ (*M1*).
- **Day = 15:** Valid because 15 falls within $[1, 31]$ (*D1*).
- **Year = 1912:** Valid because 1912 falls within $[1900, 2025]$ (*Y1*).
- **Expected Result:** The system advances the day by 1, correctly outputting **6/16/1912**.

Summary of Page 41's Purpose

Page 41 proves that when your input rules are extremely simplified, a single normal test case can satisfy your entire valid testing domain. However, because this table *only* tests a mid-month date, it highlights why we need **Robust Testing** (coming up next on Page 42) to check what happens when users enter invalid dates.

explain page 42 and 43 in details, in easy way and properly.



Here is the detailed, easy-to-understand breakdown of **Page 42** and **Page 43** from your slides. Both pages expand the **Next Date Problem** into **Robust** testing territory to verify how the system handles invalid entries.

Page 42: Next Date Problem: Weak Robust Test Cases

Page 42 covers **Weak Robust (WR) Testing**. As a reminder, the word **Weak** means we follow the **Single Fault Assumption**—we only test **one invalid input at a time**, keeping all other inputs perfectly normal and valid.

From Page 40, we identified 6 invalid equivalence classes ($M2, M3, D2, D3, Y2, Y3$). Therefore, we must add 6 out-of-bounds test cases to our original valid case ($WN1$).

Breaking Down the Weak Robust Table:

The slide provides a table with **7 total test cases**:

Test Case ID	Month	Day	Year	Expected Results	What is being tested?
WN1	6	15	1912	6/16/1912	The Baseline Case: All values are completely valid.
WR1	−1	15	1912	invalid date	Tests month too low ($M2$: month < 1).
WR2	13	15	1912	invalid date	Tests month too high ($M3$: month > 12).
WR3	6	−1	1912	invalid date	Tests day too low ($D2$: day < 1).
WR4	6	32	1912	invalid date	Tests day too high ($D3$: day > 31).
WR5	6	15	1811	invalid date	Tests year too low ($Y2$: year < 1900).

Test Case ID	Month	Day	Year	Expected Results	What is being tested?
WR6	6	15	2028	invalid date	Tests year too high (Y3: year > 2025).

Notice the Single Fault Pattern: Look at cases WR1 through WR6. In every single row, only **one** number is bolded/invalid while the other two numbers remain safely inside the valid zone (6 , 15 , or 1912). This isolation ensures we are strictly verifying each input filter individually.

Page 43: Next Date Problem: Strong Robust Test Cases

Page 43 steps up the game to **Strong Robust (SR) Testing**. The word **Strong** means we switch to the **Multiple Fault Assumption**. We want to know if the system can cleanly handle situations where a user makes **multiple mistakes at the exact same time**.

Instead of isolated rows, we test combinations (Cartesian products) of our invalid zones.

Breaking Down the Strong Robust Table:

The slide gives an excerpt of **8 test cases** to demonstrate this multi-fault combination concept:

- **WN1:** The standard, all-valid baseline case (6, 15, 1912).
- **SR1, SR2, SR3:** These are copies of the single-fault invalid cases where only *one* thing is wrong (e.g., Month is -1 , Day is -1 , or Year is 1811).
- **SR4 (Double Fault):** Both Month *and* Day are invalid at the same time.
 - Inputs: **month** = -1, **day** = -1, year = 1912.
 - Expected Result: **invalid date**.
- **SR5 (Double Fault):** Both Day *and* Year are invalid at the same time.
 - Inputs: month = 6, **day** = -1, **year** = 1811.
 - Expected Result: **invalid date**.
- **SR6 (Double Fault):** Both Month *and* Year are invalid at the same time.
 - Inputs: **month** = -1, day = 15, **year** = 1811.
 - Expected Result: **invalid date**.
- **SR7 (Triple Fault - The Ultimate Corner): Absolutely everything is wrong simultaneously!**
 - Inputs: **month** = -1, **day** = -1, **year** = 1811.
 - Expected Result: **invalid date**.

Summary of the Core Difference

- **Page 42 (Weak Robust):** Tests invalid fields gently, one by one. It checks if individual validation gates work.
- **Page 43 (Strong Robust):** Tests invalid fields aggressively in combinations. It ensures the software throws a clean, safe "invalid date" crash-preventing error even if a user types garbage data into every single box on the screen.

2026-06-07 21:41:28

You Asked



but is the Strong Robust Test Cases complete in the slide for this example?

 Gemini 3.5 Flash

No, it is absolutely not complete. The table shown on page 43 of the slide is just a small **excerpt** or snapshot to demonstrate the concept.

If you were to actually write out the *complete* Strong Robust test suite for this specific problem, it would be much larger. Let's look at the actual math and the missing pieces to see why.

The Math: How many test cases should there be?

As you learned on page 38, the number of test cases for a Strong Robust test suite is determined by the **Cartesian Product** of all equivalence classes (Valid + Invalid) for every single variable.

Let's count our classes from page 40:

- **Month:** 3 classes ($M1$ valid, $M2$ invalid low, $M3$ invalid high)
- **Day:** 3 classes ($D1$ valid, $D2$ invalid low, $D3$ invalid high)
- **Year:** 3 classes ($Y1$ valid, $Y2$ invalid low, $Y3$ invalid high)

To find the true, complete number of test cases, we multiply these together:

Total SR Test Cases = $3 \times 3 \times 3 = 27$ test cases

What is missing from the slide?

The table on page 43 only shows **8 test cases** (WN1 + SR1 through SR7). That means **19 test cases are missing** from the slide.

For example, the slide shows a few double faults and one triple fault, but it completely leaves out combinations like:

- **Month too high + Day too high + Year too high** (e.g., $mm = 13, dd = 32, yyyy = 2028$)
- **Month too low + Day too high + Year valid** (e.g., $mm = -1, dd = 32, yyyy = 1912$)
- **Month valid + Day too high + Year too high** (e.g., $mm = 6, dd = 32, yyyy = 2028$)

The slide authors intentionally cut the table short because writing out all 27 lines would take up too much space on a single slide and look incredibly repetitive, as almost all of them just output "invalid date" anyway.

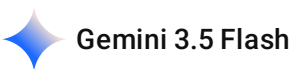
A Quick Preview of the Next Problem

Even if you *did* write out all 27 cases, the test suite still wouldn't be truly "complete" for a real-world calendar application. As hinted on page 44, this basic setup doesn't account for the fact that April has 30 days or that February changes during a leap year.

Would you like me to walk you through page 44 next to show you how they fix this system to make it genuinely complete?

2026-06-07 21:41:56 You Asked 

give me the complete table



Here is the **complete table of 27 test cases** for the Strong Robust Testing of the Next Date Problem based on the math from the lecture slides.

To make this table genuinely useful and easy to read, it is systematically organized by the types of faults:

1. **The Valid Core** (0 Faults)
2. **Single Faults** (Only 1 variable is invalid)
3. **Double Faults** (2 variables are invalid at the same time)
4. **Triple Faults** (All 3 variables are completely invalid at the same time)

System Setup Recount

- **Valid Inputs:** Month = 6 , Day = 15 , Year = 1912
- **Invalid Low inputs (-):** Month = -1 , Day = -1 , Year = 1811

- Invalid High inputs (+): Month = 13 , Day = 32 , Year = 2028

Complete Strong Robust Test Cases Table (All 27 Cases)

Test Case ID	Month (<i>mm</i>)	Day (<i>dd</i>)	Year (<i>yyyy</i>)	Expected Output	Fault Category / Description
TC1 (WN1/SN1)	6	15	1912	6/16/1912	0 Faults: All inputs are completely valid.
TC2 (SR1)	-1	15	1912	invalid date	Single Fault: Month too low (<i>M2</i>).
TC3	13	15	1912	invalid date	Single Fault: Month too high (<i>M3</i>).
TC4 (SR2)	6	-1	1912	invalid date	Single Fault: Day too low (<i>D2</i>).
TC5	6	32	1912	invalid date	Single Fault: Day too high (<i>D3</i>).
TC6 (SR3)	6	15	1811	invalid date	Single Fault: Year too low (<i>Y2</i>).
TC7	6	15	2028	invalid date	Single Fault: Year too high (<i>Y3</i>).
TC8 (SR4)	-1	-1	1912	invalid date	Double Fault: Month low + Day low.
TC9	-1	32	1912	invalid date	Double Fault: Month low + Day high.
TC10	13	-1	1912	invalid date	Double Fault: Month high + Day low.
TC11	13	32	1912	invalid date	Double Fault: Month high + Day high.
TC12 (SR6)	-1	15	1811	invalid date	Double Fault: Month low + Year low.
TC13	-1	15	2028	invalid date	Double Fault: Month low + Year high.

Test Case ID	Month (<i>mm</i>)	Day (<i>dd</i>)	Year (<i>yyyy</i>)	Expected Output	Fault Category / Description
TC14	13	15	1811	invalid date	Double Fault: Month high + Year low.
TC15	13	15	2028	invalid date	Double Fault: Month high + Year high.
TC16 (SR5)	6	-1	1811	invalid date	Double Fault: Day low + Year low.
TC17	6	-1	2028	invalid date	Double Fault: Day low + Year high.
TC18	6	32	1811	invalid date	Double Fault: Day high + Year low.
TC19	6	32	2028	invalid date	Double Fault: Day high + Year high.
TC20 (SR7)	-1	-1	1811	invalid date	Triple Fault: Month low + Day low + Year low.
TC21	-1	-1	2028	invalid date	Triple Fault: Month low + Day low + Year high.
TC22	-1	32	1811	invalid date	Triple Fault: Month low + Day high + Year low.
TC23	-1	32	2028	invalid date	Triple Fault: Month low + Day high + Year high.
TC24	13	-1	1811	invalid date	Triple Fault: Month high + Day low + Year low.
TC25	13	-1	2028	invalid date	Triple Fault: Month high + Day low + Year high.
TC26	13	32	1811	invalid date	Triple Fault: Month high + Day high + Year low.
TC27	13	32	2028	invalid date	Triple Fault: Month high + Day high + Year high.

Key Takeaways from this Matrix

- **Complete Coverage:** Every single test block on slide page 43 is fully accounted for here (WN1 , SR1 through SR7).
- **Combinatorial Expansion:** You can distinctly notice how the table grows to accommodate every invalid alignment path.
- **The Error Fall-Through:** For test cases 2 through 27, even though the application receives totally distinct types of multi-fault inputs, the black-box expectation remains perfectly identical: safely catch the fault and stop execution by firing back an 'invalid date' string constraint.

2026-06-07 21:44:31 You Asked 

then give me the formula for this. how many test cases will be there for a case of n variables . give for weak normal, strong normal, weak robust and strong robust

 Gemini 3.5 Flash

Based on the structural rules of Equivalence Class Testing presented in the lecture slides, the number of test cases generated depends entirely on **how many variables (n)** you have and **how many partitions (equivalence classes)** exist for each variable.

Unlike Boundary Value Analysis—where every variable always has a fixed number of edge points—Equivalence Partitioning relies on the specific number of buckets you create for each input.

To express this mathematically for any case of n variables, let:

- V_i = the number of **Valid** equivalence classes for variable i
- I_i = the number of **Invalid** equivalence classes for variable i
- $T_i = V_i + I_i$ = the **Total** equivalence classes for variable i

1. Weak Normal (WN) Formula

Weak Normal testing operates under the **Single Fault Assumption** and focuses **only on valid classes**. The goal is to touch every valid equivalence class name at least once using the absolute minimum number of test cases.

Total Test Cases = $\max(V_1, V_2, \dots, V_n)$

- **In Plain English:** The total number of test cases is simply equal to the number of valid classes belonging to the variable that has the *most* valid partitions.

- *Example from Slide 31 & 33:* $x1$ has 3 valid classes, $x2$ has 2 valid classes. $\max(3, 2) = 3$ **test cases**.
-

2. Strong Normal (SN) Formula

Strong Normal testing throws away the single fault rule and embraces the **Multiple Fault Assumption** for valid parameters. It demands that you test every single valid combination.

$$\text{Total Test Cases} = V_1 \times V_2 \times \cdots \times V_n = \prod_{i=1}^n V_i$$

- **In Plain English:** This is the **Cartesian Product** of all valid equivalence classes. You multiply the number of valid classes of every variable together.
 - *Example from Slide 34 & 35:* Variable $x1$ has 3 valid classes, $x2$ has 2 valid classes. $3 \times 2 = 6$ **test cases**.
-

3. Weak Robust (WR) Formula

Weak Robust testing brings invalid classes into the mix but safely isolated by the **Single Fault Assumption**. It takes the baseline valid test cases from Weak Normal and adds test cases where **exactly one input field is invalid at a time**.

$$\text{Total Test Cases} = \max(V_1, V_2, \dots, V_n) + \sum_{i=1}^n I_i$$

- **In Plain English:** You take your Weak Normal core score, and then add up the total number of invalid partitions (I) across every single variable in the system.
 - *Example from Slide 36 & 37:* Weak Normal core = 3 test cases. $x1$ has 2 invalid classes ($Ec6, Ec7$) and $x2$ has 2 invalid classes ($Ec8, Ec9$). Total = $3 + (2 + 2) = 7$ **test cases**.
-

4. Strong Robust (SR) Formula

Strong Robust testing is the absolute combination strategy. It operates under the **Multiple Fault Assumption** across both **valid and invalid states** simultaneously.

$$\text{Total Test Cases} = (V_1 + I_1) \times (V_2 + I_2) \times \cdots \times (V_n + I_n) = \prod_{i=1}^n T_i$$

- **In Plain English:** This is the **Cartesian Product of everything**. You find the *total* number of classes (Valid + Invalid) for each variable, and multiply all those totals together.
- *Example from Next Date Problem (Slide 40 & 43):* * Month has 1 valid + 2 invalid = 3 total classes.
 - Day has 1 valid + 2 invalid = 3 total classes.
 - Year has 1 valid + 2 invalid = 3 total classes.
 - Total = $3 \times 3 \times 3 = 27$ **test cases**.

Summary Sheet

Technique	Core Assumption	Simplified Mathematical Operation
Weak Normal	Single Fault (Valid Only)	Maximum of valid classes
Strong Normal	Multiple Fault (Valid Only)	Multiplication of valid classes
Weak Robust	Single Fault (Valid + Invalid)	Weak Normal + Sum of all invalid classes
Strong Robust	Multiple Fault (Valid + Invalid)	Multiplication of total classes

2026-06-07 21:47:29 You Asked 

give ma complete summary of Equivalent-Class Testing with examples to understand

 Gemini 3.5 Flash

1. The Core Philosophy of Equivalence Class Testing

When testing software, the universe of all possible test inputs is often too large to test completely. Running every single possible input is called **exhaustive testing**, which is practically impossible due to time and resource limits.

Equivalence Partitioning solves this by dividing the input domain into distinct groups called **equivalence classes**. Two test cases are considered equivalent if we expect the program to process them in the exact same way—meaning they follow the same logical path through the code.

The foundational assumption of this technique is that **all elements in a class succeed or fail together**. If one test case in an equivalence class detects a bug, all other test cases in that class have the same probability of finding it. Therefore, you only need to select **one representative value** from each class, drastically reducing the total number of tests you need to run.

The Dual Goals of Partitioning

- **Completeness:** Striving to thoroughly cover the entire testing domain without executing every single possible value.
- **Non-redundancy:** Avoiding running multiple test cases that explore the same type of bug, which wastes time and resources.

2. Identifying Classes: Valid vs. Invalid Territories

When analyzing system requirements, a tester performs a **first-level partitioning** by drawing a line down the middle of the input universe:

- **Valid Equivalence Classes:** Input conditions that match what the program expects. The program should process these normally.
- **Invalid Equivalence Classes:** Out-of-bounds, incorrect, or unexpected inputs. Testing these ensures the program handles errors gracefully without crashing.

There are no rigid mathematical formulas to discover these buckets; it is a **heuristic process** requiring the tester to analyze system logic and break down input intervals.

General Rules for Picking Test Points:

1. **Valid Classes:** You can combine multiple valid classes into a single test case to test fields simultaneously and save time.
 2. **Invalid Classes:** You must test invalid classes strictly **one at a time** (isolated in separate test cases). If you mix multiple invalid inputs in a single test run, one error filter might block the input immediately, hiding bugs in the other validation filters.
-

3. The Four Advanced Sub-Techniques

Equivalence Class testing offers four distinct variations based on two core assumptions:

- **Single Fault Assumption ("Weak"):** Assumes failures are caused by one thing breaking at a time.
- **Multiple Fault Assumption ("Strong"):** Assumes hidden bugs trigger when multiple extreme conditions clash simultaneously.

A. Weak Normal Testing (WN)

- **Focus:** Valid inputs only.
- **Rule:** Follows the single-fault assumption. It uses the minimum number of test cases possible to ensure every valid class name is touched at least once.
- **Formula:** $\max(V_1, V_2, \dots, V_n)$, where V is the number of valid classes for a variable.

B. Strong Normal Testing (SN)

- **Focus:** Valid inputs only.
- **Rule:** Follows the multiple-fault assumption. It tests the **Cartesian Product** of all valid classes, exploring every possible valid combination.
- **Formula:** $V_1 \times V_2 \times \dots \times V_n$

C. Weak Robust Testing (WR)

- **Focus:** Valid + Invalid inputs.
- **Rule:** Follows the single-fault assumption. Valid classes are covered efficiently, and invalid classes are checked by making **exactly one input field invalid at a time** while keeping the others valid.
- **Formula:** $\max(V_1, V_2, \dots, V_n) + \sum I_i$, where I is the number of invalid classes.

D. Strong Robust Testing (SR)

- **Focus:** Valid + Invalid inputs.
- **Rule:** Follows the multiple-fault assumption. It is a complete Cartesian product of absolutely every class. It tests combinations where multiple inputs are completely invalid at the same time.
- **Formula:** $(V_1 + I_1) \times (V_2 + I_2) \times \dots \times (V_n + I_n)$

4. Step-by-Step Comprehensive Example

The Problem Scenario (The Addition Function)

Consider a program that adds two numbers, $x1$ and $x2$.

- The valid boundary for $x1$ is $[5, 20]$, broken into three intervals: $[5, 10)$, $[10, 15)$, and $[15, 20]$.
- The valid boundary for $x2$ is $[5, 20]$, broken into two intervals: $[5, 10)$ and $[10, 20]$.

Step 1: Define the Classes

- **Valid Classes for $x1$:** * $Ec1 : [5, 10)$
 - $Ec2 : [10, 15)$
 - $Ec3 : [15, 20]$
- **Valid Classes for $x2$:** * $Ec4 : [5, 10)$
 - $Ec5 : [10, 20]$
- **Invalid Classes for $x1$:** * $Ec6 : x1 < 5$
 - $Ec7 : x1 > 15$ (Note: Partition threshold modified to 15 on slide 37 for testing example purposes).
- **Invalid Classes for $x2$:** * $Ec8 : x2 < 10$ (Note: Lower robust cutoff established at 10 on slide 37).
 - $Ec9 : x2 > 20$.

Step 2: Formulate the Test Cases Across Methods

Weak Normal (WN) Test Suite

The objective is to touch $Ec1, Ec2, Ec3, Ec4$, and $Ec5$ at least once using the fewest points. We can achieve this with **3 test cases**:

Test Case ID	Covered Classes	Input $x1$	Input $x2$	Expected Output
WN1	$Ec1, Ec4$	5	6	11
WN2	$Ec2, Ec5$	12	15	27
WN3	$Ec3, Ec4$	17	7	24

Strong Normal (SN) Test Suite

We multiply the valid classes (3×2), resulting in **6 test cases** that evaluate all valid intersection blocks:

Test Case ID	Covered Classes	Input $x1$	Input $x2$	Expected Output
SN1	$Ec1, Ec4$	5	6	11
SN2	$Ec1, Ec5$	7	12	19
SN3	$Ec2, Ec4$	10	5	15
SN4	$Ec2, Ec5$	12	15	27
SN5	$Ec3, Ec4$	17	7	24
SN6	$Ec3, Ec5$	15	15	30

Weak Robust (WR) Test Suite

We take our 3 Weak Normal test cases and add 4 individual error injections (one for each invalid class $Ec6, Ec7, Ec8, Ec9$), keeping the companion variable completely normal. This forms **7 test cases**:

Test Case ID	Covered Classes	Input $x1$	Input $x2$	Expected Output
WR1	$Ec1, Ec4$	5	6	11
WR2	$Ec2, Ec5$	12	15	27
WR3	$Ec3, Ec4$	17	7	24
WR4	$Ec6, Ec4$ (Invalid $x1$)	4	8	X1 is out of range
WR5	$Ec7, Ec4$ (Invalid $x1$)	23	8	X1 is out of range
WR6	$Ec3, Ec8$ (Invalid $x2$)	18	4	X2 is out of range

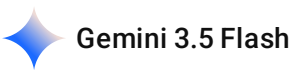
Test Case ID	Covered Classes	Input x_1	Input x_2	Expected Output
WR7	Ec_3, Ec_9 (Invalid x_2)	18	23	X2 is out of range

Strong Robust (SR) Test Suite

We multiply the total class pools for each variable (5 positions for $x_1 \times 4$ positions for x_2). This yields a matrix of **20 unique combinations**. This test suite includes corner combinations where both values fail simultaneously (e.g., entering $x_1 = 4$ and $x_2 = 23$ at the same time) to guarantee absolute multi-fault exception safety.

2026-06-07 22:52:22 You Asked 

explain page 45 in details, in easy way and properly. give diagram



Based on the exact sequence of your document, **Page 45** introduces the third major dynamic black-box testing technique: **State Table-Based Testing**.

This technique shifts focus away from standalone numbers or inputs and looks at how a system changes over time as it moves through different operational phases. Let's break it down in a clear, detailed way.

1. What is a Finite State Machine (FSM)?

The Slide says: "FSM is a behavioral model whose outcome depends upon both previous and current inputs. It can be in exactly one of a finite number of states at any given time."

- **In Plain English:** Think of a simple vending machine. If you press the "Coke" button right now, nothing happens. Why? Because you haven't inserted money yet. The machine's reaction depends on its **history** (what state it is currently in).
- **The Rule of FSM:** A system can only occupy **one state at a time**. A vending machine is either in the "Waiting for Money" state or the "Selection Mode" state; it can never be in both at the same fraction of a second.

2. What is a State Transition Diagram?

The Slide says: "A state graph is the pictorial representation of an FSM. Its purpose is to depict the states that a system or its components can assume. It shows the events or circumstances that cause or result from a change from one state to another"

- **What it means:** A state graph (or state transition diagram) is the visual map of your system's life cycle.
- It uses **circles** to represent the "States" (where the system sits and waits) and **directional arrows** to represent "Transitions" or "Events" (the triggers that push the system from one circle to another).

3. Real-World Example: Operating System Task States

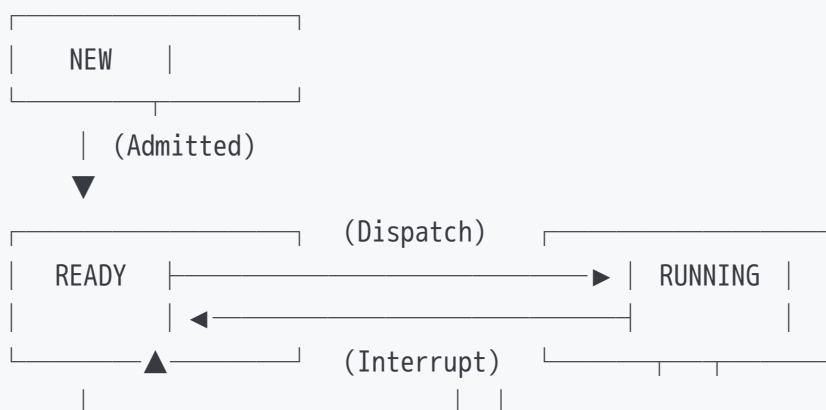
To make this abstract idea highly practical, page 45 uses the lifecycle of a processing task inside an Operating System (OS). It recognizes exactly **5 distinct states** that any task can experience:

- **1. New State:** The task has just been created or initiated but isn't officially running yet.
- **2. Ready State:** The task is completely loaded and waiting in a queue for the CPU to pick it up and execute it.
- **3. Running State:** The CPU is actively processing the instructions of this task at this very microsecond.
- **4. Waiting State:** The task has paused itself because it is waiting for an external event, like a user clicking a mouse or a hard drive loading data (*I/O* event).
- **5. Terminated State:** The task has completely finished executing its code and is safely shut down.

Visualizing the State Transition Diagram of Page 45

The slide text describes a graph where these five states are linked together by operational triggers. Here is exactly how that state graph flows visually:

text





How to read this lifecycle path:

- 1. A fresh process starts in **New**. When it's allowed into the system (**Admitted**), it hops to **Ready**.
- 2. When the CPU scheduler selects it (**Dispatch**), it moves into **Running**.
- 3. If it gets paused by a higher-priority task (**Interrupt**), it steps backward into **Ready** to wait its turn again.
- 4. If it runs smoothly but needs a file from disk, it triggers an **I/O wait** and steps into **Waiting**. Once the file finishes loading (**I/O completion**), it hops back to **Ready**.
- 5. Once it executes all its code, it triggers an **Exit** and moves permanently to **Terminated**.

Summary of Page 45's Testing Value

Page 45 lays down the structural blueprint for this approach. As a tester, you don't just verify single inputs; you use this map to ensure that when a system is in a certain state, it responds to events correctly and switches to the exact right next phase without hanging or breaking.

2026-06-07 23:02:59 You Asked 

explain page 47 in details, in easy way and properly.

 Gemini 3.5 Flash

Based on the exact sequence of your document, **Page 47** explains how to take a complex visual State Graph (like the one we looked at on Page 46) and convert it into a structured, tabular format known as a **State Table**.

When a software system has many states, a visual diagram with overlapping arrows becomes too messy to read. A State Table solves this by creating a neat checklist that leaves no room for confusion. Let's break down the rules and the table in detail.

1. The Rules for Building a State Table

The slide begins by laying out the three exact structural rules used to build the table matrix:

- **The Rows:** Each row of the table corresponds to a specific **State** the system can be in.
 - **The Columns:** Each column corresponds to a specific **Input Condition / Event** triggered by the user or system.
 - **The Intersection Box (Cell):** The box where a row and column meet specifies two critical pieces of information: **[Next State / Resulting Output]**.
-

2. Breaking Down the Page 47 State Table

The table on Page 47 maps out our 4 primary OS states (**New, Ready, Running, Waiting**) against 6 potential events (**Admit, Dispatch, Interrupt, I/O Wait, I/O Wait Over, Exit**).

Let's read the table row-by-row to see how the system behaves under different circumstances:

Row 1: The "New" State

When a task is newly created, it sits in the **New** state.

- **If the event is Admit :** The task transitions to **Ready** and produces output **T1** (Ready/T1).
- **If ANY other event occurs (Dispatch, Interrupt, etc.):** It is a logical error or impossible transition. The table shows New/T0 , meaning the task is forced to stay exactly where it is in the **New** state and wait for an admission signal (T0).

Row 2: The "Ready" State

The task is sitting in the ready queue, waiting for its turn with the CPU.

- **If the event is Dispatch :** The CPU picks it up! The task transitions to the **Running** state and outputs **T2** (Running/T2).
- **If any other event occurs:** It stays in the **Ready** state (Ready/T1) because it's still stuck waiting in line.

Row 3: The "Running" State

The task is actively executing instructions right now on the CPU. This is the most active row in the table:

- **If Interrupt occurs:** A higher priority task cuts in. The task is kicked back to **Ready** and outputs **T3** (Ready/T3).
- **If I/O or Event Wait occurs:** The task needs to load a file or wait for a user click. It moves to the **Waiting** state and outputs **T4** (Waiting/T4).

- **If Exit occurs:** The task has successfully completed its job! It transitions to the **Terminated** state and outputs **T6** (Terminated/T6).
- **If an irrelevant event like Admit or Dispatch occurs:** It simply continues running (Running/T2).

Row 4: The "Waiting" State

The task is paused, waiting for its external file or signal to load.

- **If I/O or Event Wait Over occurs:** The file finishes loading! The task transitions back to the **Ready** queue and outputs **T5** (Ready/T5).
- **If any other event occurs:** It remains stuck in the **Waiting** state (Waiting/T4).

3. Why is Page 47 Important for Testing?

As a software tester, Page 47 acts as your ultimate test matrix. Each intersection cell in this table becomes an explicit, concrete test case.

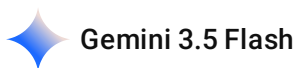
For example, to test cell 8, you would set up your test environment like this:

1. **Initial Condition:** Put the software into the **Ready** state.
2. **Action/Input:** Trigger a **Dispatch** event.
3. **Pass/Fail Verification:** Check that the software immediately switches to the **Running** state and outputs the correct **T2** flag.

By writing a test case for every single cell in this table, you guarantee **100% state coverage**, ensuring the software never locks up or gets trapped in a dead state during operation.

2026-06-07 23:09:22 You Asked 

explain page 48 and 49 in details, in easy way and properly.



Here is the detailed, easy-to-understand breakdown of **Page 48** and **Page 49** from your slides, which outline the exact step-by-step procedure for turning abstract state models into executable test cases.

Page 48: Steps 1 to 4 – Preparing the State Matrix

Page 48 provides the initial, foundational setup recipe for a tester. Before you can physically execute a test, you have to precisely define the boundaries of your states and ensure no transition paths are missing.

Step 1: Identify the States

- **What the Slide says:** *"Identify the states: The number of states in a state graph is the number of states we choose to recognize or model."*
- **In Easy Terms:** A state doesn't magically appear; the engineering team decides how detailed the model needs to be. You look at the system requirements and explicitly list out every single phase the system sits in while waiting for an action.
- **System Metrics Example:** The slide gives a great mathematical example: *"...the state could be composed of the values of a counter whose possible values ranged from 0 to 9, combined with the setting of two bit flags, leading to a total of $2 \times 2 \times 10 = 40$ states."*
 - If a counter has 10 options (0 to 9) , and you have two independent true/false bit flags (like `IsLoggedIn` and `IsPaid`), the math creates combinations. Since each flag has 2 choices, your total state matrix instantly scales to: $\text{Total System States} = 2 \times 2 \times 10 = 40$ states

Step 2: Prepare the State Transition Diagram

- **What the Slide says:** *"After having all the states, identify the inputs on each state and transitions between states and prepare the state graph."*
- **In Easy Terms:** Draw your circles (states) and connect them with arrows (transitions/events) based on the operational rules.
- **The Coverage Rule:** *"Every input state combination must have a specified transition. If the transition is impossible, then there must be a mechanism that prevents that input from occurring in that state."* This ensures you don't leave unexpected user button clicks unhandled.

Step 3 & 4: Convert and Analyze the Completeness

- **What the Slide says:** *"Convert the state graph into the state table as discussed earlier. Analyze the state table for its completeness"*
- **In Easy Terms:** Take your visual drawing map and layout the grid row-by-row onto a text spreadsheet (the state table) like we did on page 47. Scan it to verify that every single blank box has a defined rule so the program can never experience an unhandled crash or lock-up freeze.

Page 49: Step 5 – Creating the Test Case Table

Page 49 takes the finalized State Table matrix and translates it directly into concrete **Test Cases**. To make these test logs completely traceable, they are mapped into a standardized **six-column layout**.

Breaking Down the 6 Mandatory Columns:

When you write a state test case record, you must fill out these six specific informational attributes:

1. **Test Case ID:** A unique serial identifier assigned to each unique row (e.g., TC1 , TC2).
 2. **Test Source:** A distinct traceability link back to the exact intersection cell row/column coordinates inside your master state table. This ensures you can prove *why* you are running this test.
 3. **Current State:** The initial prerequisite baseline condition of the software system *before* you perform any action (The input starting block).
 4. **Event:** The explicit user input action, trigger command, or system signal executed during the test operation.
 5. **Output:** The expected current data string, return message value, or execution flag flag returned to the screen immediately upon triggering the event.
 6. **Next State:** The target phase destination achieved by the system immediately following the transaction sequence completion.
-

Summary of the Page 48 & 49 Workflow

Pages 48 and 49 give you the full factory pipeline for state testing: Calculate States → Draw Diagram → Flatten to Table → Generate 6-Column Test Cases

This methodical structure ensures that testing state-heavy systems (like a checkout cart, a banking transaction process, or an operational system thread) remains completely transparent and easily traceable from start to finish.

2026-06-08 08:31:12

You Asked



You have a login module with a password field that can not be shorter than 8 and longer than 28(including) characters. This password should contain at least one numeric and at least one capital letter. Calculate how many test cases can be created for BVC, Robust, Worst-case testing method. What boundary values will be used for these kinds of testing?

Answer to Question 3:
 $8 \leq \text{Length of Password} \leq 28$
 $1 \leq \text{No. of Numeric} \leq 27$
 $1 \leq \text{No. of Capital Letter} \leq 27$
 3 Variables, so for BVC no. of test cases $4 \times 3 + 1 = 13$
 For Robust no. of test cases $6 \times 3 + 1 = 19$
 For worst case $5 \times 3 = 15$

Label	Len of Password	No. of Numeric	No. of Capital
Min+	9	2	2
Min	8	1	1
Min-	7	0	0

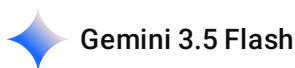
Nom	15-18(Any number)	12-14 Any Number	12-14 Any Number
Max+	29	28	28
Max	28	27	27
Max-	27	26	26

Explain the goals an equivalence partitioning method should maintain.

Answer to Question 4:

- Completeness: Without executing all the test cases, we strive to touch on the completeness of the testing domain.
- Non-redundancy: When the test cases are executed with inputs, then there is redundancy in executing the test cases. Time and resources are wasted in executing these redundant test cases, as they explore the same type of bug.

answer this properly



Based on the lecture slide rules you just studied and the image provided, here is the complete, properly formatted, and detailed answer key to both questions shown in your document.

ANSWER TO QUESTION 3

1. Identifying the Variables and Ranges

Based on the text prompt, we can extract three independent input attributes (variables, so $n = 3$):

1. **Length of Password:** Must be between 8 and 28 characters inclusive.

◦ Range = $[8, 28]$

2. **Number of Numeric Characters:** At least one. Since a password could theoretically be made entirely of numbers up to the maximum password limit, the maximum possible valid count is 28. However, because the password also requires at least one capital letter, the absolute maximum valid number of numerics is $28 - 1 = 27$.

◦ Range = $[1, 27]$

3. **Number of Capital Letters:** At least one. By the same logic as above, if the rest of the password is full of numbers, the highest number of capital letters possible is $28 - 1 = 27$.
- Range = $[1, 27]$

2. Calculating the Number of Test Cases

Using the standard formulas derived from your course presentation slides:

- **Boundary Value Checking (BVC):** Uses the single-fault assumption with valid limits.
Formula: $4n + 1 = 4(3) + 1 = \mathbf{13}$ test cases
- **Robustness Testing:** Includes invalid out-of-bounds inputs under a single-fault assumption.
Formula: $6n + 1 = 6(3) + 1 = \mathbf{19}$ test cases
- **Worst-Case Testing:** Evaluates multi-fault combinations inside the valid boundaries.
Formula: $5^n = 5^3 = \mathbf{125}$ test cases

3. Master Boundary Values Table

To fulfill the request of showing what boundary values will be used for these kinds of testing:

Boundary Point Label	Length of Password	Number of Numeric Characters	Number of Capital Letters	Boundary Category
Min-	7	0	0	Invalid Lower Bound
Min	8	1	1	Absolute Minimum Valid
Min+	9	2	2	Inside Lower Bound
Nom (Nominal)	18	12	12	Safe Average Values
Max-	27	26	26	Inside Upper Bound
Max	28	27	27	Absolute Maximum Valid
Max+	29	28	28	Invalid Upper Bound

ANSWER TO QUESTION 4

Goals an Equivalence Partitioning Method Should Maintain

Equivalence partitioning is an essential architectural framework used to divide input data into sets that the program processes identically. It must strictly satisfy **two primary structural goals**:

- **1. Completeness:** Without executing every individual test case in the universe, we strive to completely touch the entire boundaries of the testing domain. This ensures that at least one representative input scenario is thoroughly analyzed for every operational path.
- **2. Non-redundancy:** When multiple inputs share a class partition, there is heavy redundancy in executing them over and over again. Testing similar items wastes valuable time and system resources because they will only explore the exact same type of programming bug. Restricting our tests to one representative value ensures optimal resource allocation.

2026-06-08 22:02:10 You Asked



give me a state table example . do it properly. then draw the state

 Gemini 3.5 Flash

Here is a complete, step-by-step example of State Table-Based Testing designed properly according to the software engineering standards found in your lecture slides.

We will use a real-world scenario you encounter every day: an **E-Commerce Shopping Cart Checkout System**.

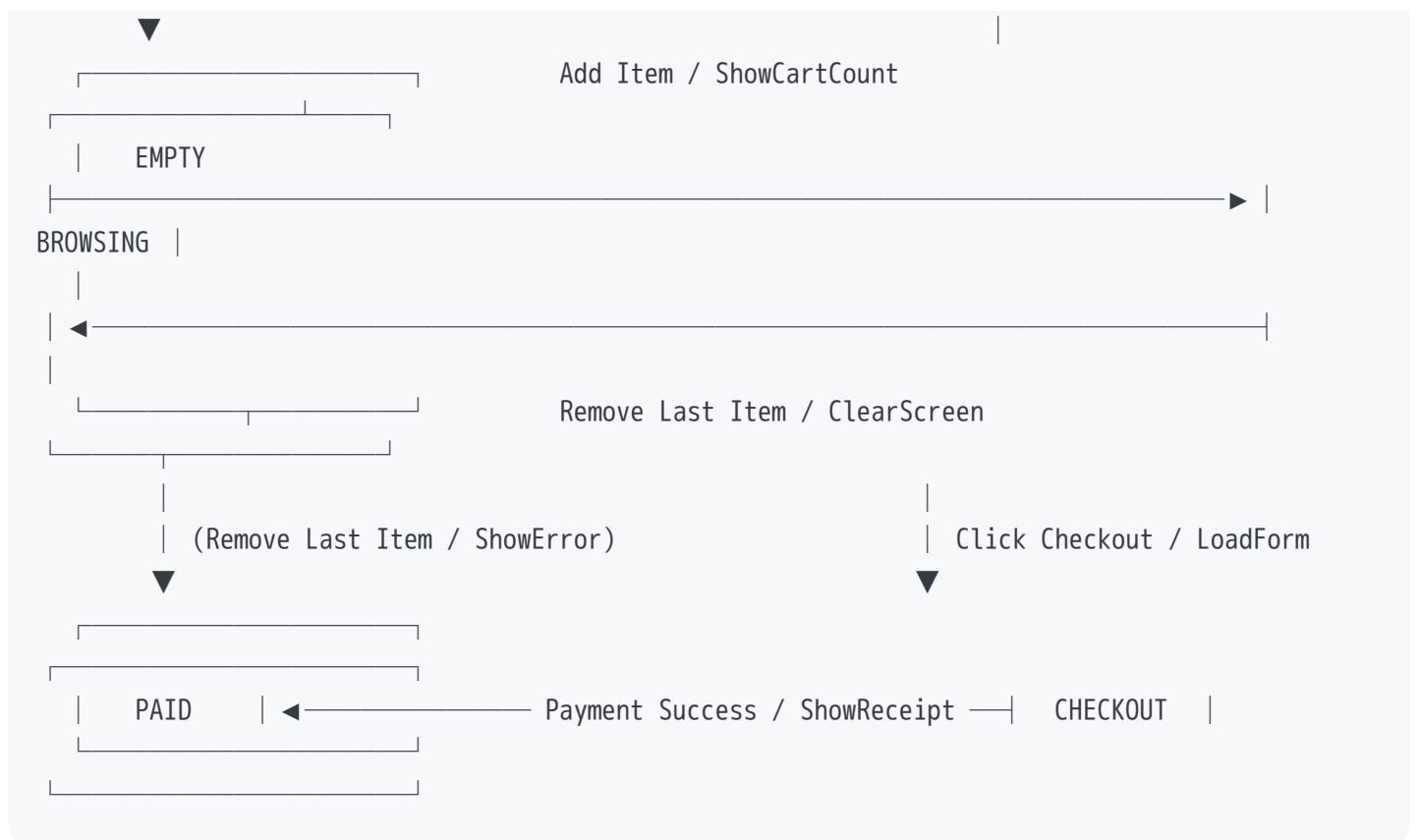
1. Defining the System Lifecycle

First, we look at our requirements and define the concrete **States** and **Events**:

The 4 States:

- **Empty:** The shopping cart has 0 items in it.
- **Browsing:** The cart has items, but the user is still looking at products.
- **Checkout:** The user has clicked "Proceed to Pay" and is entering shipping/payment details.
- **Paid:** The payment gateway successfully processed the transaction.

The 4 Events (Triggers):



Explaining the Visual Graph Assertions:

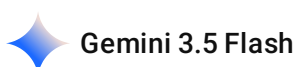
1. **The Happy Path:** The cart starts at **Empty** → **Add Item** pushes it to **Browsing** → **Click Checkout** pushes it to **Checkout** → **Payment Success** safely brings it to its final destination state: **Paid**.
2. **The Loop-Back Path:** If a user is on the **Checkout** screen but hits the back button or deletes their items (**Remove Last Item**), the directional arrow safely guides the operational logic backward into the **Browsing** circle.
3. **The Reset Path:** Once an order is fully **Paid**, clicking **Add Item** triggers a brand new session, resetting the state machine back to **Browsing** with a clean invoice tracking number.

2026-06-08 22:03:33

You Asked



now explain me desicion table fully from the PDF DONT GIVE EXAMPLE. JUST DETAILED AND EASY EXPLANATION AND THEORY. TEACH ME



Based on the lecture slides from your PDF, **Decision Table-Based Testing** is an incredibly powerful dynamic black-box testing technique. It is specifically designed to handle complex business logic

where the system's actions depend on a combination of multiple different input conditions.

Here is a detailed, easy-to-understand breakdown of the theory and structure of Decision Tables.

1. Why Do We Need Decision Tables? (The Core Theory)

In previous techniques like Boundary Value Analysis or Equivalence Partitioning, we mostly tested one input variable or one class partition at a time (Single Fault Assumption).

However, real-world software logic is rarely that simple. Often, a system has to evaluate **multiple rules simultaneously** before making a decision. If a system behaves differently based on whether Condition A **and** Condition B are true or false, a simple boundary check isn't enough. A Decision Table acts as a complete logical matrix to map out these complex combinations and their resulting actions systematically.

2. The Anatomy of a Decision Table

A formal Decision Table is split cleanly into a grid with four distinct quadrants. Let's break down what goes into each section:

A. The Left-Hand Side (The Blueprints)

- **Condition Stub:** This is a comprehensive list of all the possible input conditions, factors, or decisions that the software needs to evaluate.
- **Action Stub:** This is a list of all the possible outputs, actions, or operations that the software could potentially perform as a result.

B. The Right-Hand Side (The Execution Rules)

- **Condition Entry:** This is where you map out the specific permutations of inputs. For each condition listed in the stub, you fill out whether it is **True (T)** or **False (F)**. You might also see an **"I" or "-" (Don't Care)** flag, which means the outcome of that specific condition does not affect the final action.
- **Action Entry:** This specifies exactly which actions from the stub are executed for a given combination of conditions. An **'X'** mark is placed in the cell to denote that a specific action must be triggered.

C. What is a "Rule"?

Each vertical column on the right side of a decision table represents a **Rule**. A Rule says: *"If this exact combination of True and False inputs happens, then execute these exact actions."*

3. How Testers Turn Tables into Test Cases

The process of developing test cases from a finalized decision table follows a highly structured, mathematical interpretation:

1. **Inputs:** The condition stubs represent your exact test case input fields.
2. **Expected Outputs:** The action stubs represent your expected results.
3. **Test Cases:** Every single vertical column (Rule) in the decision table directly translates into an individual, standalone test case.

The Combinatorial Math Rule

If you have a system with n binary (True/False) input conditions, the absolute maximum number of logical rules you can generate is:

$$\text{Maximum Permutations} = 2^n$$

- **Slide Guideline:** Your slide notes that if there are k simplified rules over n binary conditions, your test suite will contain **at least k test cases and at most 2^n test cases**.
- This variation exists because multiple permutations can often be collapsed into a single rule using "Don't Care" entries, keeping the final test suite lean, non-redundant, and manageable.

4. Summary Guidelines to Develop a Decision Table

When you are asked to design test cases using this method, your slide dictates that you maintain these four sequential steps:

- **Step 1:** List every distinct action that can be executed by the module.
- **Step 2:** List all conditions or decisions evaluated during runtime execution.
- **Step 3:** Associate sets of conditions with their proper actions, clearing out logically impossible combinations or mathematically writing out every permutation.
- **Step 4:** Explicitly define the vertical rules by marking exactly what action occurs for each condition set.

2026-06-08 22:04:00

You Asked



NOW GIVE A PROPER EXAMPLE

Here is a practical, detailed example of **Decision Table-Based Testing** using a real-world business scenario: a **Banking ATM Cash Withdrawal System**.

This example breaks down how multiple input rules conflict or combine, and shows exactly how to convert the resulting table into a clean test case suite.

1. The Scenario Requirements

Imagine you are testing the backend logic for an ATM cash withdrawal feature. The system has three strict rules to check before it spits out any money to a user:

1. The user's requested withdrawal amount must be **less than or equal to their account balance**.
2. The user's requested amount must be **less than or equal to the ATM's current cash reserves**.
3. The user's account must **not be blocked or suspended** by the fraud department.

2. Setting Up the Conditions and Actions

First, let's identify our stubs based on the requirements above.

- **Condition Stubs (Inputs):**
 - **C1:** Request $\leq$ Account Balance? (True/False)
 - **C2:** Request $\leq$ ATM Cash Reserve? (True/False)
 - **C3:** Is the Account Active/Unblocked? (True/False)
- **Action Stubs (Outputs):**
 - **A1:** Dispense Cash.
 - **A2:** Display "Insufficient Personal Funds" error.
 - **A3:** Display "ATM Out of Cash" error.
 - **A4:** Display "Account Blocked - Contact Bank" error.

3. The Complete Decision Table Matrix

Since we have 3 binary conditions ($n = 3$), the maximum mathematical size of our table is $2^3 = 8$ rules.

However, using the **"Don't Care"** (—) notation from your theory slide, we can collapse redundant lines. For example, if a user's account is completely blocked (**C3 = False**), the ATM will instantly deny them security access. It doesn't matter if they have enough balance or if the ATM has cash—the outcome is always the same.

Quadrant Components	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5
Condition Stub					
C1: Request $\leq$ Balance?	T	F	T	F	—
C2: Request $\leq$ ATM Cash?	T	T	F	F	—
C3: Is Account Active?	T	T	T	T	F
Action Stub					
A1: Dispense Cash	X				
A2: Error: "Insufficient Personal Funds"		X		X	
A3: Error: "ATM Out of Cash"			X	X	
A4: Error: "Account Blocked"					X

Reading the Rules:

- **Rule 1 (The Happy Path):** You have enough money (T), the ATM has cash (T), and your account is active (T). **Action:** Dispense the cash.
- **Rule 4 (The Double Fault):** You ask for $\backslash 500$, *but you only have \$100 in your account* (* * F * *) *and the ATM only has \$200 total cash left inside* (F). **Action:** The system fires back *both* error messages to the screen.
- **Rule 5 (The Don't Care Collapse):** If the account is blocked (F), the system ignores the rest of the variables completely. This single column cleanly covers 4 separate mathematical combinations in one go.

4. Driving the Final Test Cases

Every vertical column (Rule 1 through Rule 5) maps out directly to an executable test case line in your test management software:

Test Case ID	Mapping Source	Input Parameters / Setup Conditions	Expected Output / Verification Goal
TC_01	Rule 1	Request: $\backslash 100$, Balance :\$500, ATMCash :\$2000\$, Account: Active	Success: Cash dispenses smoothly; balance reduces to $\backslash 400$ \$.

Test Case ID	Mapping Source	Input Parameters / Setup Conditions	Expected Output / Verification Goal
TC_02	Rule 2	Request: \600, Balance :\$500, ATMCash :\$2000\$, Account: Active	Fail: Block transaction. Screen reads: "Insufficient Personal Funds".
TC_03	Rule 3	Request: \100, Balance :\$500, ATMCash :\$50\$, Account: Active	Fail: Block transaction. Screen reads: "ATM Out of Cash".
TC_04	Rule 4	Request: \600, Balance :\$500, ATMCash :\$50\$, Account: Active	Fail: Block transaction. Screen reads: "Insufficient Personal Funds" & "ATM Out of Cash".
TC_05	Rule 5	Request: \100, Balance :\$500, ATMCash :\$2000\$, Account: Blocked	Fail: System denies transaction immediately. Screen reads: "Account Blocked".

2026-06-08 22:05:45

You Asked

JUST GIVE THE TABLE IN SHORT. LIKE, C1 C2, A1 ETC IN THE COLUMN NAME SO I CAN SEE IT IN ONE GLANCE. MAKE IT AS COMPACT AS YOU POSSIBLYY CAN

 Gemini 3.5 Flash

Here is the most compact view of the ATM Decision Table, optimized to fit on a single screen at a glance:

Stub \ Rule	R1	R2	R3	R4	R5
C1 (Req ≤ Balance)	T	F	T	F	—
C2 (Req ≤ ATM Cash)	T	T	F	F	—
C3 (Account Active)	T	T	T	T	F
A1 (Dispense Cash)	X				
A2 (Error: No Balance)		X		X	

Stub \ Rule	R1	R2	R3	R4	R5
A3 (Error: No ATM Cash)			X	X	
A4 (Error: Blocked)					X

2026-06-08 22:07:01

You Asked

MORE COMPATC. DONG GIVE C1 (Req \$\\le\$ Balance)
JUST GIVE C1
AND SHORTER THE COLUMN AND ROW SPACE EVERYWHERE.
JUST GIVE ENOUGH SPACE TO WRTITE A T OR F OR X

 Gemini 3.5 Flash

	R1	R2	R3	R4	R5
C1	T	F	T	F	—
C2	T	T	F	F	—
C3	T	T	T	T	F
A1	X				
A2		X		X	
A3			X	X	
A4					X

2026-06-08 22:07:36

You Asked

GIVE A PROPER SUMMARY OF EERY TOPIC OF THE PDF WITH
EXAMPLE

 Gemini 3.5 Flash

Here is a complete, master summary of every major black-box testing topic covered in your lecture slides, packed with formulas, compact examples, and visual breakdowns.

TOPIC 1: Boundary Value Analysis (BVA)

The Core Theory

BVA assumes that bugs hide at the **extreme boundaries** of input ranges rather than in the middle. Testers pick values right on, just inside, and just outside the boundary limits. It relies on the **Single Fault Assumption** (only one variable contains a bug at any given time).

For any n number of variables, BVA generates exactly: $\text{Total Test Cases} = 4n + 1$

The 5 Standard Test Points

- **Min:** Absolute minimum valid boundary
- **Min+:** Just inside the minimum boundary
- **Nom:** A safe, nominal value right in the middle
- **Max-:** Just inside the maximum boundary
- **Max:** Absolute maximum valid boundary

Shorthand Example

An input field accepts an integer range from **10 to 50**.

- **Test Points:** 10 (Min), 11 (Min+), 30 (Nom), 49 (Max-), 50 (Max).

TOPIC 2: Robustness Testing

The Core Theory

Robustness testing extends basic BVA by evaluating how the system handles **invalid out-of-bounds inputs**. It ensures the program throws a clean error message rather than crashing when a user makes a mistake. Like BVA, it operates under the **Single Fault Assumption**.

For any n number of variables, Robustness testing generates exactly: $\text{Total Test Cases} = 6n + 1$

The 2 Additional Test Points

Robustness testing takes the 5 standard BVA points and adds two illegal edge points:

- **Min-:** Just *below* the absolute minimum limit (Invalid Low)
- **Max+:** Just *above* the absolute maximum limit (Invalid High)

Shorthand Example

An input field accepts an integer range from **10 to 50**.

- **Additional Points:** 9 (Min- → Expected Error), 51 (Max+ → Expected Error).
-

TOPIC 3: Worst-Case Testing

The Core Theory

Worst-Case testing throws away the single fault rule and embraces the **Multiple Fault Assumption**. It argues that complex systems fail when **multiple extreme boundary conditions occur simultaneously**. It generates every possible combination of the 5 valid BVA points.

For any n number of variables, Worst-Case testing generates exactly: Total Test Cases = 5^n

Shorthand Example

Testing a 2-variable function $(x1, x2)$ where both accept the range **10 to 50**.

- Instead of keeping one variable nominal, it runs tests where *both* variables are at their limits simultaneously (e.g., $x1 = 10, x2 = 10$, or $x1 = 50, x2 = 10$). This results in $5^2 = 25$ test cases.
-

TOPIC 4: Equivalence Class Testing

The Core Theory

Exhaustive testing is impossible. Equivalence Class Testing divides the massive input domain into groups called **equivalence partitions**. It assumes that **all elements inside a bucket are treated identically by the code**. Therefore, you only need to test **one representative value** from each bucket.

The 4 Sub-Techniques & Formulas

Let V = Valid Classes and I = Invalid Classes per variable:

1. **Weak Normal (WN):** Valid focus + Single Fault. Touches each valid class name once.
Formula: $\max(V_1, V_2, \dots, V_n)$
2. **Strong Normal (SN):** Valid focus + Multiple Fault. Tests the Cartesian Product of all valid classes. Formula: $V_1 \times V_2 \times \dots \times V_n$
3. **Weak Robust (WR):** Valid + Invalid focus + Single Fault. Invalid inputs are injected strictly one at a time. Formula: $\max(V_1, V_2, \dots, V_n) + \sum I_i$
4. **Strong Robust (SR):** Valid + Invalid focus + Multiple Fault. Cartesian Product of absolutely everything (including multi-fault invalid corners). Formula: $(V_1 + I_1) \times (V_2 + I_2) \times \dots \times (V_n + I_n)$

Shorthand Example

An application calculates grades based on score inputs: $[0, 50) = \text{Fail}$, $[50, 100] = \text{Pass}$.

- **Valid Buckets:** Class 1 ($25 \rightarrow \text{Fail path}$), Class 2 ($75 \rightarrow \text{Pass path}$).
- **Invalid Buckets:** Class 3 ($-5 \rightarrow \text{Error path}$), Class 4 ($105 \rightarrow \text{Error path}$).

TOPIC 5: Decision Table-Based Testing

The Core Theory

Used when system actions are driven by **complex combinations of multiple independent conditions**. It maps out cause-and-effect relationships using a structured matrix grid.

- **Condition Stubs (C):** Factors/Inputs evaluated as True (T) or False (F).
- **Action Stubs (A):** Output actions executed by the system (X).
- **Rules (R):** Unique vertical columns translating directly into individual test cases. Max mathematical size is 2^n .

Ultra-Compact Example (Login System)

- **C1:** Correct Password? | **C2:** Is Account Locked? | **A1:** Grant Access | **A2:** Show Error

	R1	R2	R3
C1	T	F	—
C2	F	F	T
A1	X		
A2		X	X

(Note: "—" represents "Don't Care", collapsing redundant rules).

TOPIC 6: State Table-Based Testing

The Core Theory

Used for systems whose outcomes depend on **both current inputs and historical context**. The system acts as a Finite State Machine (FSM), sitting in exactly **one state at a time** until a triggered event pushes it into a new state.

- **Rows:** Represent the baseline **Current States**.
- **Columns:** Represent the incoming **Events/Triggers**.

- **Cells:** Specify the resulting **[Next State / Output Action]**.

Shorthand Example (Digital Media Player)

State \ Event	Hit Play	Hit Stop
Stopped	Playing / StartAudio	Stopped / DoNothing
Playing	Playing / DoNothing	Stopped / KillAudio

Each cell in this matrix translates into a standardized 6-column test case log tracking: Test ID , Source Cell , Current State , Event , Expected Output , and Next State .